

Web Developer Full-Stack desde Cero

V4 · Edición visual ampliada para principiantes absolutos

Objetivo del curso

Que puedas construir, probar, depurar y publicar una aplicación web full-stack entendiendo por qué funciona cada pieza, no solo copiando código. Al terminar deberías poder decir: "Entiendo qué estoy viendo, sé para qué sirve cada parte y puedo seguir el proceso para construirlo."

Principio didáctico

Cada tema sigue la misma ruta: ¿Qué es? → ¿Para qué sirve? → ¿Cómo funciona? → ¿Cómo se ve gráficamente? → Ejemplo cotidiano → Ejemplo en el proyecto → Código → Explicación línea por línea → Ejercicio guiado → Ejercicio independiente → Error común → Debugging → Preguntas → Aplicación real.

Ruta tecnológica:

HTML

- > CSS
- > JavaScript
- > Git/GitHub
- > HTTP/APIs
- > Node.js
- > Express
- > PostgreSQL
- > Autenticacion/Seguridad
- > Testing/Debugging
- > React
- > Calidad/Rendimiento
- > Deployment
- > Capstone

Índice

Sección	Página aproximada
Cómo estudiar este curso · Público objetivo	3
Proyecto conductor: EventHub · Mapa de proyectos	4
Módulo 0 — Mentalidad Full-Stack y mapa de la web	
Módulo 1 — Entorno profesional y terminal	
Módulo 2 — HTML semántico y accesibilidad	
Módulo 3 — CSS y responsive design	
Módulo 4 — JavaScript esencial	
Módulo 5 — DOM, eventos y formularios	
Módulo 6 — Git, GitHub y flujo de trabajo	
Módulo 7 — HTTP, JSON y APIs	
Módulo 8 — Asincronía y fetch	
Módulo 9 — Arquitectura de frontend	
Módulo 10 — Node.js y backend	
Módulo 11 — Express y REST API	
Módulo 12 — SQL y PostgreSQL	
Módulo 13 — Persistencia: Express + PostgreSQL	
Módulo 14 — Autenticación y autorización	
Módulo 15 — Seguridad web básica	
Módulo 16 — Testing y debugging	
Módulo 17 — Frontend moderno con React	
Módulo 18 — Calidad, accesibilidad y rendimiento	
Módulo 19 — Deployment y proyecto de portafolio	
Módulo 20 — Capstone EventHub Full-Stack	
Laboratorio de debugging general	
Evaluación del aprendizaje · Checklist de graduación	

Glosario para principiantes

Examen final de transferencia

Cómo estudiar este curso

- 1 Lee primero la explicación con tus propias palabras.
- 2 Ejecuta el ejemplo y cambia al menos una cosa.
- 3 Resuelve el ejercicio sin mirar la solución completa.
- 4 Explica el concepto a otra persona o en una nota, y di por qué funciona.
- 5 Comete errores de forma deliberada y aprende a encontrarlos.
- 6 No avances por tiempo: avanza cuando puedas demostrar la habilidad.
- 7 Guarda evidencia en Git: commits, README, capturas, tests o una demo.

Regla de dominio: no consideramos aprendido un tema porque viste un video o contestaste una pregunta de memoria. Lo consideramos aprendido cuando puedes realizar la tarea y luego resolver una variante nueva con poca o ninguna ayuda.

¿Para quién es este curso?

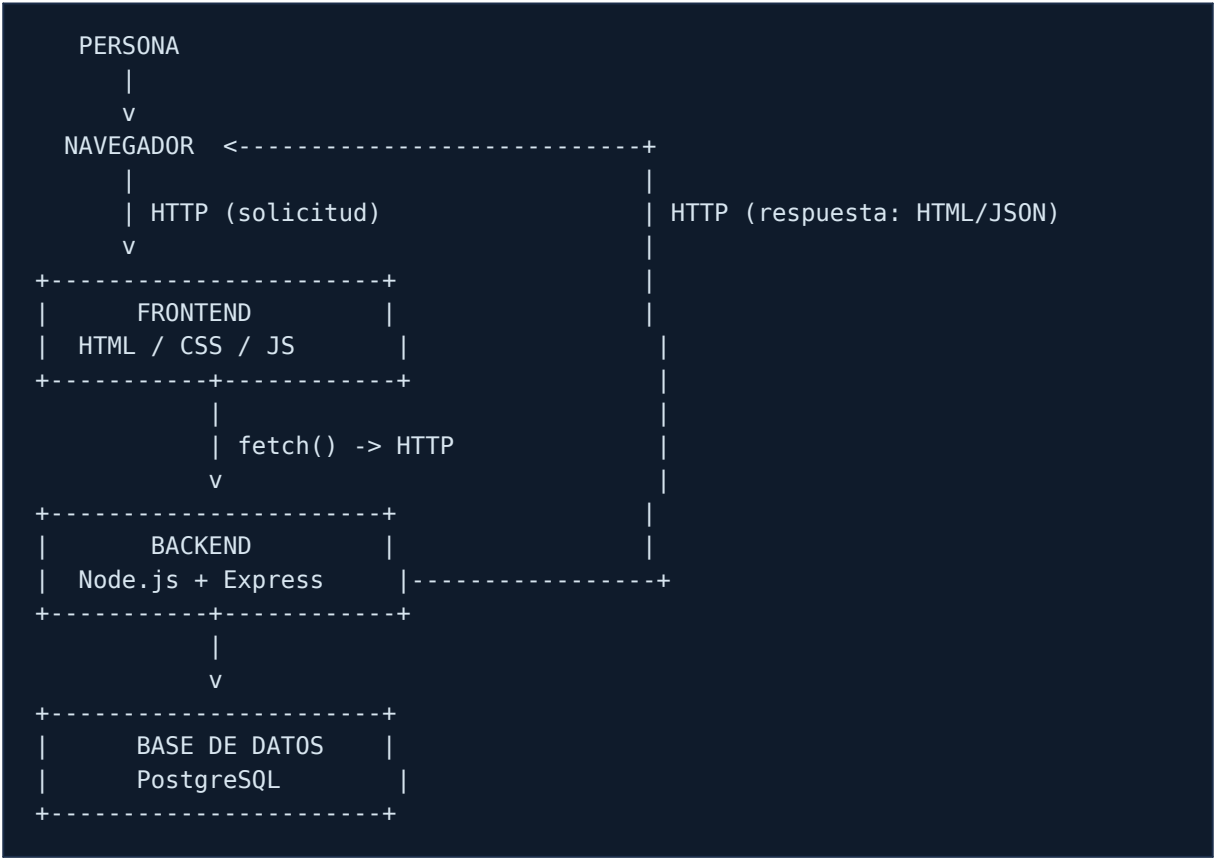
Este curso asume que:

- Nunca ha programado.
- No conoce HTML, CSS ni JavaScript.
- Nunca ha usado Git.
- No sabe qué es un servidor ni una API.
- No entiende bases de datos.
- Puede sentirse intimidado por el código.

Por eso, nunca vas a leer algo como "ahora crea una API RESTful utilizando Express" sin que antes se te haya explicado, con una analogía cotidiana, qué es una API, qué problema resuelve y cómo se ve dentro del proyecto.

Proyecto conductor: EventHub

Durante todo el curso construirás EventHub, una plataforma sencilla para gestionar eventos. El proyecto empieza como una página HTML sencilla y termina como una aplicación full-stack con usuarios, roles, base de datos, API, pruebas y deployment.



Meta final: poder explicar ese recorrido de memoria y modificar cada capa sin depender de un tutorial paso a paso.

Mapa de proyectos

Proyecto	Qué demuestra
1. Perfil personal	HTML, enlaces, imágenes, semántica y Git
2. Landing responsive	CSS, Flexbox/Grid y responsive design
3. Calculadora de entradas	JavaScript: funciones, condicionales y arrays
4. EventHub local	DOM, formularios, eventos y fetch
5. API de eventos	Node.js, Express, rutas y manejo de errores
6. Persistencia	SQL, PostgreSQL y queries parametrizadas

7. Auth + roles	Hashing, login, autorización y middleware
8. EventHub final	Integración, React, testing, seguridad y deployment

MÓDULO 0

Mentalidad Full-Stack y mapa de la web

A. Lo que vas a aprender

Vas a entender que una página web no es un archivo suelto: es un sistema con varias piezas que colaboran. Al terminar podrás dibujar y explicar ese sistema completo.

B. ¿Qué es?

Cuando visitas una web, tu navegador le pide información a una computadora que está en algún lugar de internet (un servidor). Esa computadora procesa la petición, puede consultar datos guardados, y devuelve una respuesta que tu navegador convierte en la página que ves. "Full-stack" significa aprender las piezas principales de ese recorrido: la interfaz (frontend), el servidor que decide qué hacer (backend) y el lugar donde se guardan los datos (base de datos).

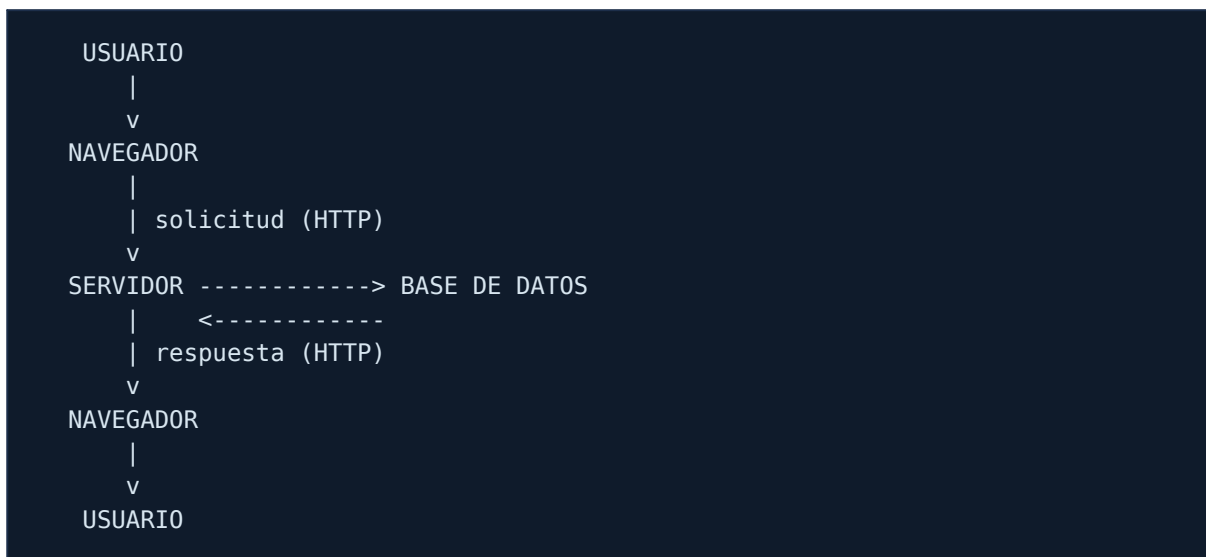
C. ¿Por qué existe?

Si no entiendes el recorrido completo, cada tecnología nueva parece un bloque aislado y mágico. Entender el mapa completo desde el primer día te permite ubicar cada herramienta futura (HTML, CSS, Express, PostgreSQL, React...) dentro de un lugar concreto del sistema, en vez de memorizar nombres sueltos.

D. ¿Cómo funciona? (paso a paso)

- 1 Escribes una dirección (URL) o haces clic en un botón.
- 2 El navegador arma una solicitud y la envía por internet.
- 3 Un servidor recibe la solicitud y decide qué hacer con ella.
- 4 Si hace falta, el servidor consulta o modifica una base de datos.
- 5 El servidor arma una respuesta (HTML, o datos en formato JSON).
- 6 El navegador recibe la respuesta y actualiza lo que ves en pantalla.

E. Diagrama visual



F. Ejemplo cotidiano (analogía)

Piensa en un restaurante. El frontend es el salón y el menú que ve el cliente. El backend es la cocina: recibe el pedido y decide qué hacer con él. La base de datos se parece a la despensa y los registros del restaurante. HTTP es el camino por el que viaja el pedido del salón a la cocina, y la respuesta de la cocina de vuelta al salón.

G. Ejemplo dentro del proyecto EventHub

En EventHub, la plataforma de eventos que construirás durante todo el curso, cuando alguien pulse "Ver evento", el navegador pedirá los datos de ese evento, el backend los buscará en la base de datos y la información aparecerá en pantalla sin que el usuario necesite saber nada de lo que ocurrió detrás.

H. Código mínimo

TEXT

```

Usuario -> Navegador -> HTTP -> Servidor
-> Base de datos -> Servidor -> Navegador
  
```

I. Explicación línea por línea

Usuario -> Navegador	La persona interactúa con la interfaz: hace clic, escribe, navega.
Navegador -> HTTP	El navegador convierte esa acción en una solicitud con un formato estándar.
HTTP -> Servidor	La solicitud viaja por internet hasta la computadora que atiende esa dirección.

Servidor -> Base de datos -> Servidor	El servidor consulta los datos que necesita para responder.
---------------------------------------	---

Servidor -> Navegador	La respuesta regresa y el navegador la convierte en algo visible.
-----------------------	---

J. ¿Qué debería ver en pantalla?

Nada de código todavía: en este módulo el resultado esperado es un diagrama propio, dibujado a mano o en texto, que puedas explicar en voz alta sin mirar el original.

K. Error frecuente

△ Pensar que "la página web" es solo lo que se ve (el HTML), olvidando que detrás hay un servidor y, casi siempre, una base de datos.

L. Cómo encontrar el error

Pregúntate: si apagara el servidor ahora mismo, ¿la página seguiría funcionando igual? Si la respuesta es no, ya sabes que hay más piezas además del HTML.

M. Ejercicio guiado

Dibuja el recorrido de una solicitud desde que el usuario pulsa "Ver evento" hasta que la información del evento aparece en pantalla. Usa flechas y nombra cada caja (navegador, servidor, base de datos).

N. Ejercicio independiente

Elige una app que uses a diario (una red social, un banco, un mapa) y dibuja su recorrido completo sin ver el ejemplo del curso.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Qué parte del sistema ve directamente el usuario?

Respuesta de referencia: El frontend: la interfaz dentro del navegador.

[COMPRENSIÓN] ¿Qué parte puede cambiar sin tocar la base de datos?

Respuesta de referencia: El frontend: se puede rediseñar la interfaz sin modificar cómo se guardan los datos.

[APLICACIÓN] ¿Qué parte necesita guardar información de forma permanente?

Respuesta de referencia: La base de datos, porque el servidor y el navegador no conservan datos entre visitas por sí solos.

[DIAGNÓSTICO] Un compañero dice: "mi página no necesita backend porque todo el HTML ya está escrito". ¿Qué le preguntarías?

Respuesta de referencia: Le preguntarías si esa página necesita guardar datos, mostrar información distinta por usuario, o procesar formularios; si no, puede que tenga razón para ese caso concreto.

[TRANSFERENCIA] ¿Cómo cambiaría el diagrama si la app no necesitara guardar ningún dato (por ejemplo, una calculadora)?

Respuesta de referencia: Desaparecería la base de datos, y quizás incluso el servidor si todo el cálculo ocurre en el navegador.

P. Mini reto

Explica con tus propias palabras, en un mensaje de voz o nota de menos de 1 minuto, qué es full-stack, como si se lo explicarás a alguien que jamás ha usado una computadora.

Q. Checkpoint

✓ **¿Puedes dibujar el diagrama completo (usuario, navegador, servidor, base de datos) sin mirar tus notas y explicar qué hace cada flecha?**

MÓDULO 1

Entorno profesional y terminal

A. Lo que vas a aprender

Vas a preparar tu "taller" de trabajo: el editor de código, la terminal y Git. Al terminar podrás crear una carpeta de proyecto y moverte por ella con confianza.

B. ¿Qué es?

El editor de código (usaremos VS Code) es donde escribes y organizas tus archivos. La terminal es una forma de hablarle directamente a tu computadora escribiendo comandos, en vez de hacer clic en íconos. Git es un programa que registra los cambios de tu proyecto a lo largo del tiempo, y GitHub es un servicio en internet donde puedes guardar copias de esos proyectos.

C. ¿Por qué existe?

Sin terminal, muchas herramientas del desarrollo web (instalar paquetes, ejecutar un servidor, usar Git) simplemente no funcionan igual de bien. Sin Git, perder un archivo o un cambio importante significa perderlo para siempre. Aprender esto primero evita frustraciones más adelante.

D. ¿Cómo funciona? (paso a paso)

- 1 Abres la terminal integrada en VS Code (o la terminal del sistema).
- 2 Escribes un comando y presionas Enter.
- 3 La terminal ejecuta ese comando y muestra el resultado como texto.
- 4 Te mueves entre carpetas con comandos, sin usar el mouse.
- 5 Git observa la carpeta y puede registrar cualquier cambio que hagas en ella.

E. Diagrama visual

```
CARPETA DEL PROYECTO (tu taller)
|
+-- VS Code          -> mesa de trabajo (donde escribes)
+-- Terminal         -> panel de control (donde das ordenes)
+-- Git (dentro de .git) -> maquina del tiempo (guarda versiones)
+-- GitHub (en internet) -> copia remota del taller
```

F. Ejemplo cotidiano (analogía)

La carpeta del proyecto es tu taller. VS Code es tu mesa de trabajo. La terminal es el panel de control del taller. Git es una máquina del tiempo que guarda fotografías del taller en distintos momentos. GitHub es una bodega externa donde guardas copias de esas fotografías por si tu taller se incendia.

G. Ejemplo dentro del proyecto EventHub

Crearás la carpeta `eventhub`, que será el hogar de todo el proyecto que construirás durante el curso, y la inicializarás como repositorio Git desde el primer día.

H. Código mínimo

BASH

```
mkdir eventhub
cd eventhub
code .
git init
```

I. Explicación línea por línea

<code>mkdir eventhub</code>	Crea una carpeta nueva llamada eventhub (make directory).
<code>cd eventhub</code>	Entra a esa carpeta (change directory).
<code>code .</code>	Abre la carpeta actual (".") en VS Code.
<code>git init</code>	Convierte la carpeta en un repositorio Git, listo para registrar cambios.

J. ¿Qué debería ver en pantalla?

VS Code se abre mostrando la carpeta eventhub vacía, y la terminal no muestra ningún error tras `git init` (suele decir algo como "Initialized empty Git repository").

K. Error frecuente

⚠ Ejecutar comandos sin fijarte en qué carpeta estás, y terminar creando archivos en el lugar equivocado.

L. Cómo encontrar el error

Escribe `pwd` (Mac/Linux) o revisa la ruta que muestra la terminal para confirmar en qué carpeta estás antes de crear o modificar archivos.

M. Ejercicio guiado

Crea la carpeta eventhub, ábrela en VS Code, crea un archivo llamado notas.txt, escribe una línea de texto y guárdalo.

N. Ejercicio independiente

Crea una segunda carpeta de práctica, muévete a ella desde la terminal sin usar el mouse, e inicialízala también como repositorio Git.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Qué diferencia hay entre una carpeta de proyecto y un repositorio Git?

Respuesta de referencia: Una carpeta solo organiza archivos; un repositorio Git además guarda el historial de cambios de esos archivos.

[COMPRENSIÓN] ¿Para qué sirve la terminal si ya existe el explorador de archivos?

Respuesta de referencia: Permite ejecutar herramientas y comandos que muchas veces no tienen equivalente visual, de forma más rápida y precisa.

[APLICACIÓN] ¿Qué comando usarías para crear una carpeta llamada `landing`?

Respuesta de referencia: mkdir landing

[DIAGNÓSTICO] Ejecutas `git init` y la terminal dice "command not found". ¿Qué revisarías primero?

Respuesta de referencia: Si Git está instalado correctamente en tu computadora.

[TRANSFERENCIA] Si ya tienes una carpeta creada de antes (sin usar mkdir), ¿qué comando necesitas para convertirla en repositorio?

Respuesta de referencia: Solo necesitas entrar a ella con cd y ejecutar git init; mkdir no es necesario si la carpeta ya existe.

P. Mini reto

Sin ver el ejemplo, crea una carpeta llamada `landing`, entra a ella, ábrela en VS Code e inicialízala con Git, en ese orden.

Q. Checkpoint

✓ ¿Puedes explicar, sin mirar tus notas, qué hace cada uno de los cuatro comandos (mkdir, cd, code ., git init)?

MÓDULO 2

HTML semántico y accesibilidad

A. Lo que vas a aprender

Vas a construir la estructura de una página usando HTML, y a que esa estructura tenga significado, no solo apariencia. Al terminar podrás crear una tarjeta de evento completa.

B. ¿Qué es?

HTML (HyperText Markup Language) describe qué es cada parte de una página: un título, un párrafo, una imagen, un botón. No decide colores ni distribución (eso es CSS); describe significado y estructura. Una etiqueta (tag) marca el inicio y el final de un elemento, y un atributo agrega información extra a ese elemento.

C. ¿Por qué existe?

El navegador, los buscadores y las personas que usan lectores de pantalla necesitan saber qué es cada parte de la página para poder mostrarla, indexarla o leerla en voz alta correctamente. Un HTML con significado (semántico) hace todo eso posible; un HTML hecho solo de cajas genéricas (`div`) no comunica nada de eso.

D. ¿Cómo funciona? (paso a paso)

- 1 El navegador lee el archivo HTML de arriba hacia abajo.
- 2 Cada etiqueta le indica al navegador qué tipo de contenido sigue.
- 3 El navegador construye una representación interna de la página (el DOM, que verás en el Módulo 5).
- 4 Esa representación se dibuja en pantalla siguiendo el orden del documento.

E. Diagrama visual

<code><main></code>	ZONA PRINCIPAL
<code><article></code>	UNA PIEZA DE CONTENIDO
<code><h2>Titulo</h2></code>	ENCABEZADO
<code><p>Texto...</p></code>	PARRAFO
<code><button>Ver</button></code>	ACCION
<code></article></code>	
<code></main></code>	

F. Ejemplo cotidiano (analogía)

Si una casa tuviera solo cajas sin etiquetas, sería difícil saber cuál es la cocina o el dormitorio. Las etiquetas semánticas de HTML son esas etiquetas de las cajas: `header`, `nav`, `main`, `section`, `article` y `footer` explican el papel de cada zona de la casa.

G. Ejemplo dentro del proyecto EventHub

Construirás la primera tarjeta de evento de EventHub: título, fecha, ciudad, imagen y un botón "Ver evento", usando etiquetas semánticas en vez de `div` genéricos.

H. Código mínimo

HTML

```
<main>
  <article>
    <h2>Festival</h2>
    <p>CDMX - 25 de agosto</p>
    <button>Ver evento</button>
  </article>
</main>
```

I. Explicación línea por línea

<main>	Marca la zona principal de contenido de la página (debe haber solo una por página).
<article>	Marca una pieza de contenido independiente: en este caso, un evento.
Festival	Encabezado de nivel 2: el título del evento.
CDMX - 25 de agosto	Un párrafo con información adicional del evento.
<button>Ver evento</button>	Un botón real, que el teclado y los lectores de pantalla reconocen como acción.

J. ¿Qué debería ver en pantalla?

Al abrir el archivo .html en el navegador, verás el título "Festival", el texto "CDMX - 25 de agosto" y un botón con la etiqueta "Ver evento", uno debajo del otro, sin ningún estilo todavía (letra por defecto del navegador).

K. Error frecuente

⚠ Usar `` para absolutamente todo, incluyendo títulos y botones, y olvidar los atributos `alt` en las imágenes o los `label` en los campos de formulario.

L. Cómo encontrar el error

Abre las herramientas de desarrollador del navegador (clic derecho -> Inspeccionar) y revisa el árbol de elementos: si ves muchos `div` anidados sin ninguna etiqueta semántica, es una señal de alerta.

M. Ejercicio guiado

Crea una tarjeta de evento con título, fecha, ciudad, imagen (usa un `alt` descriptivo) y botón. Agrega también un formulario pequeño con un campo de nombre y su `label`

asociado.

N. Ejercicio independiente

Construye tu página de perfil personal con ``header``, ``nav``, ``main`` y ``footer``, sin ver ningún ejemplo del curso.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Por qué un ``h1`` no es lo mismo que un ``div`` con texto grande?

Respuesta de referencia: Porque ``h1`` expresa un encabezado y tiene significado semántico; un ``div`` solo es un contenedor genérico sin significado.

[COMPRENSIÓN] ¿Qué hace el atributo ``alt`` en una imagen?

Respuesta de referencia: Describe el contenido de la imagen para lectores de pantalla y para cuando la imagen no carga.

[APLICACIÓN] Tienes tres secciones de contenido independientes en una página. ¿Qué etiqueta usarías para cada una?

Respuesta de referencia: ``<article>``, si cada sección tiene sentido por sí sola fuera de contexto.

[DIAGNÓSTICO] Un formulario no reacciona cuando haces clic en la etiqueta de texto junto al campo. ¿Qué falta probablemente?

Respuesta de referencia: El atributo ``for`` en el ``label``, que debe coincidir con el ``id`` del ``input``.

[TRANSFERENCIA] ¿Cómo estructurarías semánticamente una página de blog con varios artículos?

Respuesta de referencia: ``main`` para el contenido general, y un ``article`` por cada entrada del blog, cada uno con su propio ``h2`` o ``h3``.

P. Mini reto

Construye una tarjeta de "perfil de organizador" (nombre, foto, biografía corta, botón "Contactar") usando solo etiquetas semánticas, sin ver el ejemplo del evento.

Q. Checkpoint

✓ ¿Puedes explicar, sin mirar tus notas, la diferencia entre un elemento, una etiqueta y un atributo?

MÓDULO 3

CSS y responsive design

A. Lo que vas a aprender

Vas a darle apariencia, orden y adaptabilidad a la interfaz. Al terminar podrás construir una cuadrícula de eventos que se adapta a distintos tamaños de pantalla.

B. ¿Qué es?

CSS (Cascading Style Sheets) decide cómo se ven y se distribuyen los elementos HTML: colores, tamaños, espacios y posición. Un selector indica a qué elemento(s) aplica un estilo, una propiedad indica qué característica cambiar (color, tamaño...) y un valor indica cómo cambiarla.

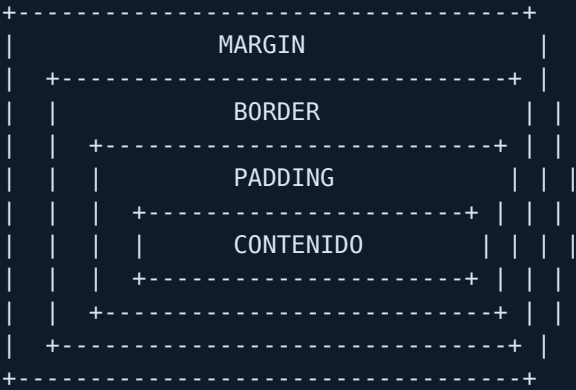
C. ¿Por qué existe?

Separar estructura (HTML) de apariencia (CSS) permite cambiar el diseño completo de una página sin tocar su contenido, y reutilizar el mismo diseño en muchas páginas. Sin CSS, cada página web se vería con la tipografía y el espaciado por defecto del navegador.

D. ¿Cómo funciona? (paso a paso)

- 1 El navegador lee el HTML y construye la estructura de la página.
- 2 El navegador lee el CSS y calcula qué estilos aplican a cada elemento.
- 3 Cada elemento se dibuja como una caja rectangular (box model): contenido, padding, borde y margen.
- 4 Flexbox y Grid deciden cómo se organizan esas cajas entre sí.
- 5 Las media queries cambian esas reglas según el tamaño de la pantalla.

E. Diagrama visual



RESPONSIVE:

Escritorio: [1][2][3]

```
Tablet:      [ 1 ][ 2 ]
```

```
Movl:    [ 1 ]
```

HTML construye las habitaciones de una casa; CSS decide paredes, colores, muebles y señalización. Flexbox sirve para organizar objetos en una sola dirección (como sillas en una fila). Grid sirve para organizar en filas y columnas (como un tablero de ajedrez).

Tomarás la lista de tarjetas de evento de EventHub y la organizarás en una cuadrícula (Grid) de 3 columnas en escritorio, que se convierte en 1 columna en pantallas pequeñas.

CSS

```
.events {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1rem;
}

@media (max-width: 700px) {
  .events { grid-template-columns: 1fr; }
}
```

```
.events {
```

Selector: aplica el estilo a cualquier elemento con `class="events"`.

<code>display: grid;</code>	Convierte ese elemento en una cuadrícula (Grid) para organizar a sus hijos.
<code>grid-template-columns: repeat(3, 1fr);</code>	Crea 3 columnas de ancho igual (1fr = una fracción del espacio disponible).
<code>gap: 1rem;</code>	Deja un espacio fijo entre cada tarjeta.
<code>@media (max-width: 700px) {</code>	Esta regla solo aplica cuando la pantalla mide 700px o menos.
<code>grid-template-columns: 1fr;</code>	En pantallas pequeñas, usa una sola columna.

J. ¿Qué debería ver en pantalla?

En una pantalla ancha verás las tarjetas de evento organizadas en 3 columnas parejas; al achicar la ventana del navegador por debajo de 700px, se reacomodan en una sola columna, una debajo de otra.

K. Error frecuente

⚠ Diseñar solo pensando en la pantalla de escritorio y probar el resultado en el celular hasta el final, cuando ya es más difícil corregir el diseño.

L. Cómo encontrar el error

Usa el modo "responsive" de las herramientas de desarrollador del navegador para simular distintos tamaños de pantalla mientras programas, no solo al final.

M. Ejercicio guiado

Haz que una lista de seis eventos tenga 3 columnas en escritorio, 2 en tablet (entre 700px y 1000px) y 1 en móvil (menos de 700px).

N. Ejercicio independiente

Construye una barra de navegación con Flexbox que muestre los enlaces en fila en escritorio y los apile en columna en móvil.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Qué problema resuelve ``padding`` y en qué se diferencia de ``margin``?

Respuesta de referencia: Padding es espacio interior del elemento (entre el borde y el contenido); margin es espacio exterior, entre el elemento y los que lo rodean.

[COMPRENSIÓN] ¿Cuándo conviene usar Flexbox en vez de Grid?

Respuesta de referencia: Cuando necesitas organizar elementos en una sola dirección (fila o columna); Grid conviene cuando necesitas filas y columnas a la vez.

[APLICACIÓN] Quieres que 4 tarjetas se acomoden en 2 filas de 2 columnas. ¿Qué propiedad usarías?

Respuesta de referencia: grid-template-columns: repeat(2, 1fr); en un contenedor con display: grid.

[DIAGNÓSTICO] Una tarjeta se ve pegada a las demás sin ningún espacio, aunque pusiste `margin`. ¿Qué revisarías?

Respuesta de referencia: Si el margin se aplicó al elemento correcto (selector correcto) y si algún otro estilo lo está sobrescribiendo.

[TRANSFERENCIA] ¿Cómo cambiarías el ejemplo para que la cuadrícula tenga 4 columnas en pantallas muy anchas (más de 1200px)?

Respuesta de referencia: Agregando otra media query con min-width: 1200px que use grid-template-columns: repeat(4, 1fr).

P. Mini reto

Sin ver el ejemplo, crea una cuadrícula de 2 columnas para una galería de imágenes que se convierta en 1 columna en móvil.

Q. Checkpoint

✓ **¿Puedes explicar de memoria el box model completo (contenido, padding, borde, margen) usando un objeto real como ejemplo?**

MÓDULO 4

JavaScript esencial

A. Lo que vas a aprender

Vas a aprender a expresar reglas y resolver problemas con código. Al terminar podrás escribir una función que calcule un descuento según reglas.

B. ¿Qué es?

JavaScript es el lenguaje que permite que la página piense y reaccione. Una variable guarda un valor (un número, un texto...). Una condición decide entre caminos distintos según se cumpla o no algo. Un bucle repite una acción varias veces. Una función agrupa una acción que puede reutilizarse las veces que quieras.

C. ¿Por qué existe?

HTML y CSS son estáticos: muestran siempre lo mismo. JavaScript agrega lógica: calcular precios, validar formularios, decidir qué mostrar según una condición. Sin JavaScript, una página no puede reaccionar a lo que hace la persona.

D. ¿Cómo funciona? (paso a paso)

- 1 El navegador ejecuta el código JavaScript en orden, de arriba hacia abajo.
- 2 Las variables guardan valores en la memoria mientras el programa corre.
- 3 Cuando el programa llega a un `if`, evalúa la condición y sigue por una rama u otra.
- 4 Cuando el programa llama a una función, "salta" a ejecutar ese bloque y regresa con un resultado.

E. Diagrama visual

```
PROBLEMA: calcular precio segun edad

  entrada: edad, precio
    |
    v
¿edad < 12? --si--> precio * 0.5
    | no
    v
¿edad < 18? --si--> precio * 0.8
    | no
    v
  precio completo
```

F. Ejemplo cotidiano (analogía)

Una receta de cocina tiene ingredientes, pasos y reglas. Las variables son los ingredientes, las funciones son recetas reutilizables ("hacer arroz" se puede usar en muchos platos), y las condiciones son instrucciones del tipo "si el arroz está listo, apaga el fuego".

G. Ejemplo dentro del proyecto EventHub

Escribirás la función que calculará el precio final de una entrada de EventHub según la edad de la persona: niños pagan la mitad, adolescentes pagan un 20% menos.

H. Código mínimo

JAVASCRIPT

```
function priceForAge(age, price) {  
  if (age < 12) return price * 0.5;  
  if (age < 18) return price * 0.8;  
  return price;  
}  
  
console.log(priceForAge(15, 500));
```

I. Explicación línea por línea

<code>function priceForAge(age, price) {</code>	Define una función llamada <code>priceForAge</code> que recibe dos datos: edad y precio.
<code> if (age < 12) return price * 0.5;</code>	Si la edad es menor a 12, devuelve la mitad del precio y termina ahí.
<code> if (age < 18) return price * 0.8;</code>	Si no entró en el caso anterior y la edad es menor a 18, devuelve el 80% del precio.
<code> return price;</code>	Si ninguna condición anterior se cumplió, devuelve el precio completo.
<code>console.log(priceForAge(15, 500));</code>	Llama a la función con edad 15 y precio 500, y muestra el resultado en la consola.

J. ¿Qué debería ver en pantalla?

En la consola del navegador o de Node aparecerá el número 400 (porque 15 años cae en la regla del 80% de 500).

K. Error frecuente

⚠ Copiar una función sin entender qué recibe (parámetros) y qué devuelve (return), y luego no saber por qué el resultado no es el esperado.

L. Cómo encontrar el error

Agrega `console.log` antes de cada `return` para ver qué valor tiene `age` y `price` en cada paso, y compara con lo que esperabas.

M. Ejercicio guiado

Crea una función que reciba edad y precio y calcule el descuento correspondiente para una entrada, siguiendo exactamente las reglas del ejemplo.

N. Ejercicio independiente

Escribe una función que reciba un array de precios de entradas y devuelva la suma total, sin ver ningún ejemplo similar.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Qué diferencia hay entre guardar un dato en una variable y ejecutar una función?

Respuesta de referencia: Una variable guarda un valor fijo; una función representa una acción o cálculo que puede ejecutarse muchas veces con distintos datos.

[COMPRENSIÓN] ¿Qué hace la palabra `return` dentro de una función?

Respuesta de referencia: Detiene la función y entrega un valor como resultado.

[APLICACIÓN] ¿Cómo llamarías a la función `priceForAge` para un adulto de 30 años con precio 200?

Respuesta de referencia: `priceForAge(30, 200)`, que devolvería 200 porque no cumple ninguna condición de descuento.

[DIAGNÓSTICO] La función siempre devuelve el precio completo, sin aplicar descuentos. ¿Qué revisarías primero?

Respuesta de referencia: Si las condiciones (`<12`, `<18`) están escritas correctamente y si `age` realmente está llegando con el valor esperado.

[TRANSFERENCIA] ¿Cómo agregarías una nueva regla de 10% de descuento para mayores de 65 años?

Respuesta de referencia: Agregando otro `if (age > 65) return price * 0.9;` antes del `return price;` final.

P. Mini reto

Escribe una función que calcule el precio de un boleto de cine aplicando un descuento del 30% los miércoles, sin ver ningún ejemplo del curso.

Q. Checkpoint

✓ **¿Puedes explicar en voz alta, sin ver tus notas, qué son variables, condiciones, funciones y arrays, con un ejemplo cotidiano para cada uno?**

MÓDULO 5

DOM, eventos y formularios

A. Lo que vas a aprender

Vas a conectar JavaScript con lo que el usuario ve y hace. Al terminar podrás agregar una tarjeta nueva a la página sin recargarla.

B. ¿Qué es?

El DOM (Document Object Model) es la representación de la página que el navegador mantiene en memoria mientras la muestra. JavaScript puede buscar elementos dentro de esa representación, leer su contenido y modificarlo. Un evento es algo que ocurre en la página: un clic, escribir en un campo, enviar un formulario, y JavaScript puede "escuchar" esos eventos y reaccionar.

C. ¿Por qué existe?

Sin el DOM, JavaScript no tendría forma de "tocar" lo que ya existe en la página. Sin eventos, JavaScript no sabría cuándo actuar: necesita saber que el usuario hizo clic en un botón exacto, en un momento exacto, para reaccionar ahí.

D. ¿Cómo funciona? (paso a paso)

- 1 El navegador construye el DOM a partir del HTML al cargar la página.
- 2 JavaScript usa `document.querySelector`` (u otros métodos) para encontrar un elemento del DOM.
- 3 JavaScript registra un "escucha" (`addEventListener``) sobre ese elemento para un tipo de evento.
- 4 Cuando el evento ocurre, el navegador ejecuta la función asociada.
- 5 Esa función puede leer datos, cambiar texto, agregar o quitar elementos del DOM.

E. Diagrama visual

```
HTML (plano) --navegador construye--> DOM (maqueta viva en memoria)
                                         |
                                         JavaScript busca un elemento
                                         |
                                         addEventListener('click', fn)
                                         |
                                         usuario hace click
                                         |
                                         se ejecuta fn()
                                         |
                                         el DOM cambia -> la pantalla cambia
```

F. Ejemplo cotidiano (analogía)

El HTML es el plano de una casa; el DOM es la maqueta viva de esa casa que existe en memoria mientras la visitas. JavaScript puede entrar a esa maqueta y mover piezas (cambiar un texto, agregar un mueble) cada vez que ocurre un evento, como tocar el timbre.

G. Ejemplo dentro del proyecto EventHub

Harás que, al llenar el formulario "Nuevo evento" de EventHub y pulsar "Guardar", aparezca una tarjeta nueva en la lista sin recargar la página.

H. Código mínimo

JAVASCRIPT

```
const button = document.querySelector('#save');
button.addEventListener('click', () => {
  document.querySelector('#message').textContent = 'Guardado';
});
```

I. Explicación línea por línea

<code>const button = document.querySelector('#save');</code>	Busca en el DOM el elemento con id="save" y lo guarda en la variable button.
<code>button.addEventListener('click', () => {</code>	Registra una función que se ejecutará cada vez que se haga clic en button.
<code>document.querySelector('#message').textContent = 'Guardado';</code>	Busca el elemento con id="message" y cambia su texto a "Guardado".
<code>});</code>	Cierra la función que reacciona al evento click.

J. ¿Qué debería ver en pantalla?

Al hacer clic en el botón con id="save", el texto del elemento con id="message" cambia a la palabra "Guardado" al instante, sin recargar la página.

K. Error frecuente

⚠ Cambiar el DOM sin verificar primero que el `querySelector` encontró realmente el elemento (si el selector está mal escrito, devuelve `null` y el código falla).

L. Cómo encontrar el error

Usa `console.log(button)` justo después del `querySelector`: si aparece `null`, el selector no coincide con ningún elemento del HTML.

M. Ejercicio guiado

Crea un formulario que añada una nueva tarjeta de evento a la página sin recargarla, usando ``preventDefault()`` para evitar el envío por defecto del formulario.

N. Ejercicio independiente

Crea un botón "Eliminar" en cada tarjeta que la quite de la pantalla al hacer clic, sin ver ningún ejemplo similar.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Por qué ``preventDefault()`` es útil en un formulario?

Respuesta de referencia: Evita el comportamiento por defecto del navegador, por ejemplo el recargo de página al enviar un formulario.

[COMPRENSIÓN] ¿Qué diferencia hay entre el HTML de un archivo y el DOM?

Respuesta de referencia: El HTML es el texto original del archivo; el DOM es la representación viva en memoria, que puede cambiar mientras la página está abierta.

[APLICACIÓN] ¿Qué método usarías para encontrar el primer elemento con `class="card"`?

Respuesta de referencia: `document.querySelector('.card')`

[DIAGNÓSTICO] Haces clic en el botón y no pasa nada, ni siquiera un error en consola. ¿Qué revisarías primero?

Respuesta de referencia: Si el `addEventListener` realmente se ejecutó (por ejemplo si el script se cargó antes de que existiera el botón en el HTML).

[TRANSFERENCIA] ¿Cómo mostrarías un mensaje de error si el campo de nombre está vacío al enviar el formulario?

Respuesta de referencia: Leyendo el value del campo antes de continuar, y si está vacío, cambiando el texto de un elemento de mensaje en vez de agregar la tarjeta.

P. Mini reto

Crea un contador que sume 1 cada vez que se hace clic en un botón, mostrando el número actualizado en pantalla, sin ver ningún ejemplo del curso.

Q. Checkpoint

✓ **¿Puedes explicar de memoria la diferencia entre HTML, DOM y JavaScript, y cómo se conectan mediante eventos?**

MÓDULO 6

Git, GitHub y flujo de trabajo

A. Lo que vas a aprender

Vas a aprender a guardar versiones de tu proyecto y a recuperar cambios anteriores con seguridad. Al terminar podrás hacer commits y revisar tu historial.

B. ¿Qué es?

Git no es lo mismo que GitHub. Git vive dentro de tu proyecto y registra cambios con el tiempo. GitHub es un servicio en internet donde puedes guardar ese repositorio y compartirlo con otras personas. Un commit es una fotografía de tu proyecto en un momento dado, acompañada de un mensaje que explica qué cambió.

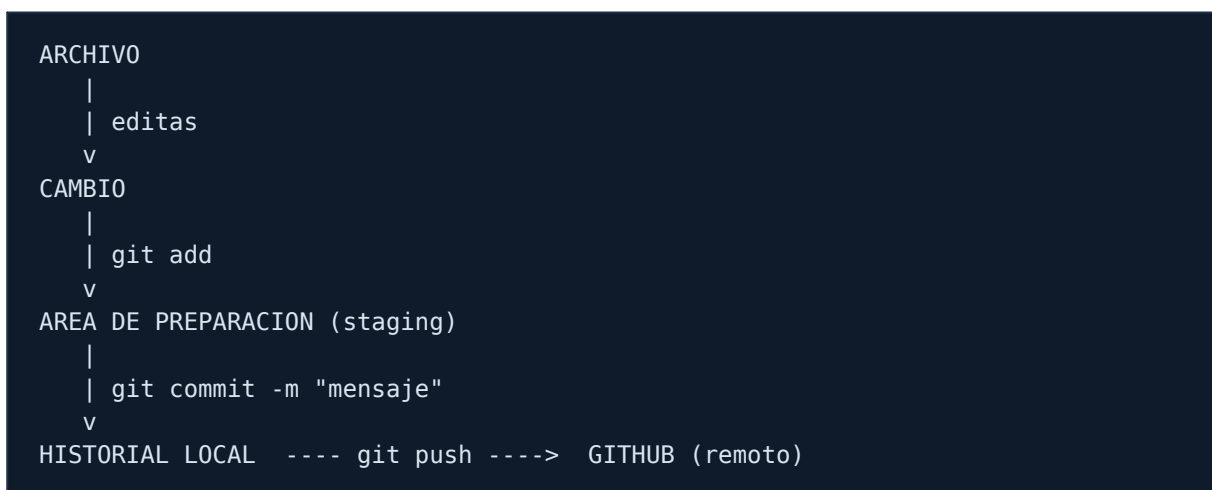
C. ¿Por qué existe?

Sin Git, si rompes algo que funcionaba, no hay forma sencilla de volver atrás. Git te permite experimentar con confianza, porque siempre puedes regresar a una versión anterior que sabías que funcionaba. GitHub además permite compartir y respaldar tu trabajo fuera de tu computadora.

D. ¿Cómo funciona? (paso a paso)

- 1 Modificas archivos dentro de tu proyecto.
- 2 Le dices a Git cuáles cambios quieres guardar (``git add``).
- 3 Creas una fotografía de esos cambios con un mensaje (``git commit``).
- 4 Git guarda esa fotografía en el historial del proyecto.
- 5 Cuando quieres, envías ese historial a GitHub (``git push``) para respaldarlo o compartirlo.

E. Diagrama visual



F. Ejemplo cotidiano (analogía)

Git es un historial de versiones, como el "deshacer" infinito de un editor, pero organizado en fotografías con nombre. GitHub es una estantería externa donde guardas y compartes ese historial, como una copia de seguridad en la nube.

G. Ejemplo dentro del proyecto EventHub

Registrarás en Git cada etapa importante de EventHub: la primera versión en HTML, la llegada de CSS, la primera función JavaScript, etc., con mensajes de commit claros.

H. Código mínimo

BASH

```
git add .
git commit -m "Agrega tarjetas de eventos"
git log --oneline
```

I. Explicación línea por línea

<code>git add .</code>	Prepara todos los archivos modificados en la carpeta actual para el próximo commit.
<code>git commit -m "Agrega tarjetas de eventos"</code>	Crea una fotografía (commit) con un mensaje que describe el cambio.
<code>git log --oneline</code>	Muestra el historial de commits, uno por línea, en forma resumida.

J. ¿Qué debería ver en pantalla?

Después de ``git log --oneline``, verás una lista con un código corto y el mensaje de cada commit que has hecho, del más reciente al más antiguo.

K. Error frecuente

⚠ Hacer un solo commit gigante al final del proyecto con el mensaje "cambios", lo que hace imposible saber qué cambió y cuándo.

L. Cómo encontrar el error

Revisa ``git log --oneline``: si ves muy pocos commits con mensajes poco descriptivos, es momento de empezar a hacer commits más pequeños y frecuentes.

M. Ejercicio guiado

Haz tres commits con cambios pequeños (por ejemplo: agregar HTML, agregar CSS, corregir un texto) y usa ``git log`` para observar el historial resultante.

N. Ejercicio independiente

Modifica un archivo, deshaz el cambio antes de hacer commit usando Git, y confirma que el archivo volvió a su estado anterior.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Por qué conviene hacer commits pequeños y descriptivos?

Respuesta de referencia: Porque facilita entender el historial, revisar cambios y localizar problemas, o volver a un estado concreto si algo se rompe.

[COMPRENSIÓN] ¿Cuál es la diferencia entre Git y GitHub?

Respuesta de referencia: Git es la herramienta que registra el historial de cambios en tu proyecto; GitHub es el servicio en internet donde puedes guardar y compartir ese historial.

[APLICACIÓN] Acabas de terminar una función nueva que funciona. ¿Qué comandos usarías en orden?

Respuesta de referencia: git add ., luego git commit -m "mensaje descriptivo".

[DIAGNÓSTICO] Ejecutas `git commit` y Git dice que no hay nada que confirmar ("nothing to commit"). ¿Qué probablemente falta?

Respuesta de referencia: Ejecutar `git add` antes, para preparar los cambios.

[TRANSFERENCIA] Si quieres compartir tu proyecto con un compañero, ¿qué pasos adicionales a los de este módulo necesitarías investigar?

Respuesta de referencia: Crear un repositorio en GitHub y usar `git push` para subir tu historial local a ese repositorio remoto.

P. Mini reto

Haz cinco commits pequeños en un proyecto de práctica, cada uno con un mensaje que describa exactamente qué cambió, y revisa el resultado con `git log --oneline`.

Q. Checkpoint

✓ ¿Puedes explicar de memoria qué hace cada uno de estos comandos: `git init`, `git add`, `git commit`, `git log`?

MÓDULO 7

HTTP, JSON y APIs

A. Lo que vas a aprender

Vas a entender cómo viajan las solicitudes y los datos entre el navegador y un servidor. Al terminar podrás diseñar en papel una API completa.

B. ¿Qué es?

HTTP es un protocolo de comunicación: un conjunto de reglas que navegadores y servidores usan para entenderse. Una solicitud (request) indica qué se quiere hacer, sobre qué recurso y con qué datos. Una respuesta (response) incluye un estado, encabezados y normalmente un cuerpo con información. JSON es un formato de texto muy común para transportar esos datos. Un endpoint es una dirección específica de una API diseñada para una operación concreta.

C. ¿Por qué existe?

El navegador y el servidor son programas distintos, quizás en computadoras distintas. Necesitan un lenguaje común y predecible para pedir y entregar información: eso es HTTP. Una API es la "puerta de entrada" que un backend expone para que otros programas (como tu frontend) puedan pedirle cosas de forma ordenada.

D. ¿Cómo funciona? (paso a paso)

- 1 El cliente (navegador) arma una solicitud: método, dirección, y a veces datos.
- 2 La solicitud viaja por internet hasta el servidor correcto.
- 3 El servidor identifica el endpoint solicitado y ejecuta la lógica correspondiente.
- 4 El servidor arma una respuesta con un código de estado y, normalmente, datos en JSON.
- 5 El cliente recibe la respuesta y decide qué hacer con ella (mostrarla, guardarla, etc.).

E. Diagrama visual



F. Ejemplo cotidiano (analogía)

Una API es como una ventanilla de atención con reglas claras: no entras a la cocina directamente, haces una solicitud siguiendo un formato conocido ("quiero el evento número 15") y recibes una respuesta también en un formato conocido.

G. Ejemplo dentro del proyecto EventHub

Diseñarás en papel la API de EventHub: qué endpoint sirve para listar eventos, cuál para ver uno en detalle, cuál para crear, actualizar o eliminar.

H. Código mínimo

TEXT

```
GET    /eventos
GET    /eventos/15
POST   /eventos
PUT    /eventos/15
DELETE /eventos/15
```

I. Explicación línea por línea

GET /eventos	Pide la lista completa de eventos (leer, sin modificar nada).
GET /eventos/15	Pide el detalle del evento con id 15.
POST /eventos	Crea un evento nuevo con los datos enviados en el cuerpo de la solicitud.
PUT /eventos/15	Reemplaza los datos del evento con id 15 con los nuevos datos enviados.
DELETE /eventos/15	Elimina el evento con id 15.

J. ¿Qué debería ver en pantalla?

Todavía no hay servidor real: el resultado esperado en este módulo es una lista clara, escrita a mano o en un documento, de cada endpoint con su método HTTP y su propósito.

K. Error frecuente

△ Confundir un endpoint de API con una página visual (por ejemplo, pensar que `/eventos` siempre devuelve una página HTML, cuando en una API suele devolver JSON).

L. Cómo encontrar el error

Pregúntate: ¿esta dirección devuelve algo para que lo vea una persona (HTML) o algo para que lo use un programa (JSON)? Si es lo segundo, es un endpoint de API.

M. Ejercicio guiado

Diseña sobre papel la API de eventos con endpoints para listar, ver el detalle, crear, actualizar y eliminar, indicando el método HTTP de cada uno.

N. Ejercicio independiente

Diseña la API de un sistema de inscripciones a eventos (relación entre usuarios y eventos), sin ver ningún ejemplo similar.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Qué diferencia hay entre una URL y un endpoint de una API?

Respuesta de referencia: Una URL identifica una dirección o recurso en general; un endpoint es una ruta de API diseñada específicamente para una operación concreta.

[COMPRENSIÓN] ¿Para qué sirve JSON en el contexto de una API?

Respuesta de referencia: Es un formato de texto que permite representar datos estructurados (objetos, listas) de forma que tanto humanos como programas puedan leerlo fácilmente.

[APLICACIÓN] ¿Qué método y endpoint usarías para eliminar el evento con id 8?

Respuesta de referencia: DELETE /eventos/8

[DIAGNÓSTICO] Un endpoint `POST /eventos` responde con éxito pero el evento no aparece luego en `GET /eventos`. ¿Qué sospecharías?

Respuesta de referencia: Que el POST no está guardando realmente el dato en ningún lugar persistente, solo simula el éxito.

[TRANSFERENCIA] ¿Qué endpoints necesitarías para que un usuario pueda inscribirse y luego cancelar su inscripción a un evento?

Respuesta de referencia: Algo como POST /eventos/:id/inscripciones para inscribirse y DELETE /eventos/:id/inscripciones/:usuarioid para cancelar.

P. Mini reto

Diseña en papel los endpoints necesarios para una API de "comentarios" en cada evento (listar, crear, eliminar un comentario), sin ver ningún ejemplo del curso.

Q. Checkpoint

✓ ¿Puedes explicar de memoria qué representa cada uno de los métodos GET, POST, PUT y DELETE?

MÓDULO 8

Asincronía y fetch

A. Lo que vas a aprender

Vas a aprender por qué el navegador puede seguir funcionando mientras espera datos de internet. Al terminar podrás consumir una API real desde JavaScript.

B. ¿Qué es?

Algunas tareas tardan: pedir datos a internet, leer un archivo, consultar un servidor. JavaScript puede esperar esos resultados sin congelar toda la interfaz gracias a la asincronía. `fetch` es una función que permite hacer solicitudes HTTP desde el navegador. Una promesa (promise) representa un valor que estará disponible en el futuro; `async` y `await` son una forma más legible de trabajar con promesas.

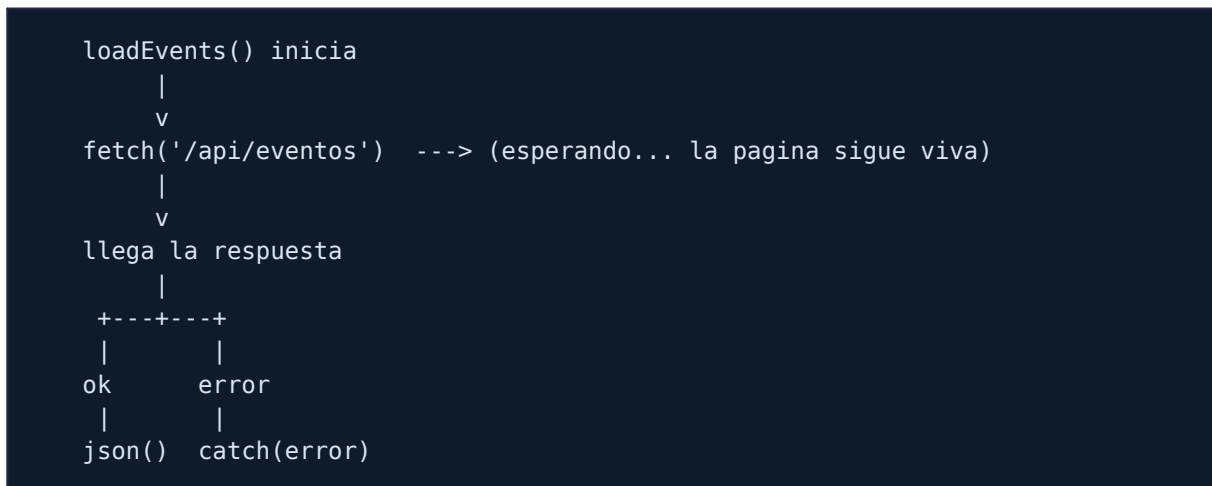
C. ¿Por qué existe?

Si el navegador se detuviera por completo cada vez que pide datos a internet, la página quedaría congelada mientras espera, y eso sería una pésima experiencia. La asincronía permite seguir interactuando con la página mientras la respuesta viaja de vuelta.

D. ¿Cómo funciona? (paso a paso)

- 1 El código llama a `fetch(url)`, que empieza la solicitud pero no bloquea el resto del programa.
- 2 Mientras se espera la respuesta, el navegador sigue funcionando con normalidad.
- 3 Cuando la respuesta llega, el código que sigue después de `await` se ejecuta con el resultado.
- 4 Si algo falla (sin conexión, error del servidor), el bloque `catch` se encarga de manejarlo.

E. Diagrama visual



F. Ejemplo cotidiano (analogía)

No necesitas quedarte parado mirando la puerta mientras esperas a un repartidor. Puedes seguir con otras cosas y recibir una notificación cuando llegue. `await` es ese "seguir con lo mío, pero avísame cuando llegue".

G. Ejemplo dentro del proyecto EventHub

Harás que EventHub pida la lista real de eventos a una API (por ahora pública, de práctica) y la muestre en pantalla, con un mensaje de carga y uno de error.

H. Código mínimo

JAVASCRIPT

```
async function loadEvents() {
  try {
    const response = await fetch('/api/eventos');
    if (!response.ok) throw new Error('Error HTTP');
    return await response.json();
  } catch (error) {
    console.error(error);
  }
}
```

I. Explicación línea por línea

<code>async function loadEvents() {</code>	Declara una función asíncrona: puede usar `await` dentro de ella.
<code> try {</code>	Inicia un bloque donde vigilamos si ocurre algún error.
<code> const response = await fetch('/api/eventos');</code>	Pide los datos y espera la respuesta antes de continuar.
<code> if (!response.ok) throw new Error('Error HTTP');</code>	Si la respuesta indica un error (por ejemplo 404 o 500), lanza un error manualmente.
<code> return await response.json();</code>	Convierte el cuerpo de la respuesta de texto a un objeto/array de JavaScript y lo devuelve.
<code> } catch (error) {</code>	Si algo falló en el bloque try, este código se ejecuta con el error capturado.
<code> console.error(error);</code>	Muestra el error en la consola para poder investigarlo.

J. ¿Qué debería ver en pantalla?

Si la API responde correctamente, la función devuelve un array de eventos; si hay un problema de red o el servidor falla, verás el error impreso en la consola en vez de que la

página se rompa en silencio.

K. Error frecuente

△ Olvidar manejar errores de red, asumiendo que ``fetch`` siempre va a funcionar, lo que hace que la aplicación se rompa sin explicación cuando el servidor falla o no hay conexión.

L. Cómo encontrar el error

Abre la pestaña Network de las herramientas de desarrollador: revisa el código de estado de la solicitud (200, 404, 500...) y compáralo con lo que esperabas.

M. Ejercicio guiado

Consume una API pública real y muestra los resultados en pantalla. Agrega un mensaje de "Cargando..." mientras esperas la respuesta y un mensaje de error si algo falla.

N. Ejercicio independiente

Agrega un botón "Reintentar" que vuelva a llamar a ``loadEvents()`` si la primera solicitud falló, sin ver ningún ejemplo similar.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Por qué una aplicación debe manejar ``loading`` y ``error`` al pedir datos?

Respuesta de referencia: Porque una petición externa puede tardar o fallar, y la interfaz debe informar al usuario mientras espera o si ocurre un problema.

[COMPRENSIÓN] ¿Qué hace la palabra ``await``?

Respuesta de referencia: Pausa la ejecución de la función (sin bloquear el resto de la página) hasta que la promesa se resuelva.

[APLICACIÓN] ¿Cómo comprobarías si una respuesta de ``fetch`` fue exitosa antes de usar sus datos?

Respuesta de referencia: Revisando la propiedad `response.ok`, o el `response.status`, antes de llamar a `response.json()`.

[DIAGNÓSTICO] La función ``loadEvents`` nunca entra al bloque ``catch``, pero tampoco muestra datos. ¿Qué revisarías?

Respuesta de referencia: Si ``response.ok`` realmente se está evaluando, y si el error ocurre después del `try/catch` (por ejemplo, al usar los datos más adelante).

[TRANSFERENCIA] ¿Cómo adaptarías este patrón para enviar un nuevo evento con POST en vez de solo leer con GET?

Respuesta de referencia: Agregando `method: 'POST'`, headers con `Content-Type`, y body: `JSON.stringify(datos)` a las opciones de `fetch`.

P. Mini reto

Escribe una función asíncrona que pida el clima de una ciudad a una API pública y muestre un mensaje de error claro si la ciudad no existe, sin ver ningún ejemplo del curso.

Q. Checkpoint

✓ ¿Puedes explicar de memoria, con tus propias palabras, qué problema resuelve la asincronía y por qué `fetch` la necesita?

MÓDULO 9

Arquitectura de frontend

A. Lo que vas a aprender

Vas a aprender a separar responsabilidades para que la interfaz pueda crecer sin volverse un caos. Al terminar podrás dividir tu código en funciones que hacen una sola cosa cada una.

B. ¿Qué es?

Cuando una app crece, conviene separar datos, presentación y lógica en piezas distintas. Una función que solo pide datos, una que solo los transforma y una que solo los muestra son más fáciles de entender, probar y corregir que una única función gigante que hace todo a la vez.

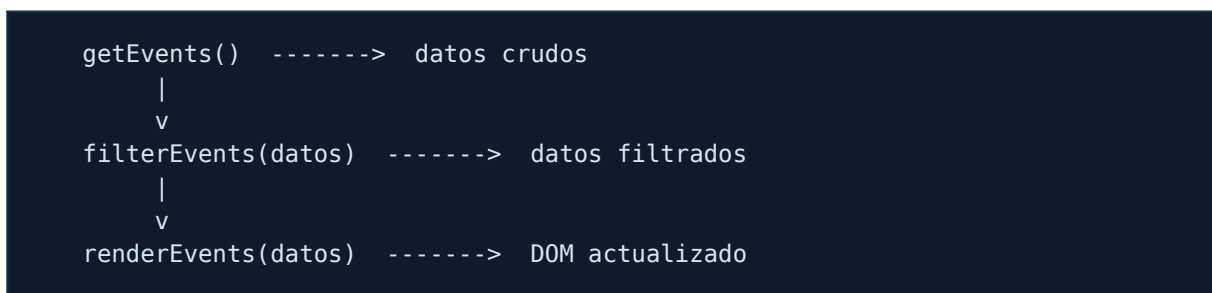
C. ¿Por qué existe?

Sin separación, cualquier cambio pequeño (por ejemplo, cambiar de dónde vienen los datos) obliga a tocar código que también dibuja la pantalla, y es fácil romper algo sin darse cuenta. Separar responsabilidades reduce el acoplamiento: cada pieza puede cambiar sola.

D. ¿Cómo funciona? (paso a paso)

- 1 Una función se encarga solo de pedir datos (por ejemplo, a una API).
- 2 Otra función, si hace falta, transforma esos datos (filtra, ordena, calcula).
- 3 Otra función se encarga solo de dibujar esos datos en el DOM.
- 4 El código que las conecta es corto y solo coordina el orden en que se llaman.

E. Diagrama visual



F. Ejemplo cotidiano (analogía)

Una cocina profesional separa almacén, preparación, cocción y salida. Si todo ocurriera en una sola mesa, con una sola persona, cualquier cambio en el menú sería muy difícil de coordinar.

G. Ejemplo dentro del proyecto EventHub

Refactorizarás el JavaScript de EventHub para que `getEvents` solo pida datos, y `renderEvents` solo los muestre, sin mezclar ambas responsabilidades en una función.

H. Código mínimo

JAVASCRIPT

```
async function getEvents() {
  const response = await fetch('/api/eventos');
  return response.json();
}

function renderEvents(events) {
  // solo presenta datos, no los pide ni los modifica
}
```

I. Explicación línea por línea

<code>async function getEvents() {</code>	Esta función se dedica exclusivamente a pedir los datos.
<code> const response = await fetch('/api/eventos');</code>	Hace la solicitud a la API.
<code> return response.json();</code>	Devuelve los datos ya convertidos, sin tocar el DOM en ningún momento.
<code>function renderEvents(events) {</code>	Esta función solo recibe datos ya listos y los muestra en pantalla.

J. ¿Qué debería ver en pantalla?

El comportamiento visible de la página no cambia, pero el código es más corto de leer por partes: puedes entender qué hace `getEvents` sin necesidad de leer cómo se dibuja la pantalla.

K. Error frecuente

⚠ Mezclar la petición de datos, las reglas de negocio y el dibujo en el DOM dentro de una sola función gigante, lo que hace muy difícil encontrar errores o reutilizar código.

L. Cómo encontrar el error

Lee el nombre de cada función y pregúntate si hace exactamente lo que su nombre promete. Si `renderEvents` también hace un `fetch`, el nombre está mintiendo sobre lo que hace.

M. Ejercicio guiado

Refactoriza tu frontend para separar la capa que pide datos de la capa que los muestra, siguiendo el patrón `getEvents / renderEvents`.

N. Ejercicio independiente

Agrega una función `filterEventsByCity(events, city)`` que se encargue solo de filtrar, sin pedir datos ni tocar el DOM, y conéctala entre las otras dos.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Qué ventaja tiene una función que solo transforma datos frente a una que también modifica el DOM?

Respuesta de referencia: Reduce el acoplamiento y hace más fácil probar, mantener y reutilizar cada parte por separado.

[COMPRENSIÓN] ¿Por qué se dice que estas funciones están "menos acopladas"?

Respuesta de referencia: Porque un cambio en una (por ejemplo, cambiar de dónde vienen los datos) no obliga a modificar las demás.

[APLICACIÓN] Si quisieras probar que `filterEventsByCity`` funciona bien, ¿necesitarías una API real corriendo?

Respuesta de referencia: No, porque esa función solo recibe datos y los transforma; puedes probarla con datos inventados.

[DIAGNÓSTICO] Cada vez que cambias el diseño visual de una tarjeta, terminas rompiendo también cómo se piden los datos. ¿Qué señala eso?

Respuesta de referencia: Que probablemente las responsabilidades de pedir datos y dibujar están mezcladas en la misma función.

[TRANSFERENCIA] ¿Cómo aplicarías esta misma idea de separación a una función que calcula el precio final de una entrada?

Respuesta de referencia: Separando la función que calcula el precio (solo matemáticas) de la que muestra ese precio formateado en pantalla.

P. Mini reto

Toma cualquier función que hayas escrito antes que haga más de una cosa a la vez, y divídela en dos funciones más pequeñas con nombres claros.

Q. Checkpoint

✓ ¿Puedes explicar de memoria por qué separar responsabilidades facilita encontrar errores?

MÓDULO 10

Node.js y backend

A. Lo que vas a aprender

Vas a entender y ejecutar JavaScript fuera del navegador. Al terminar podrás levantar tu primer servidor local.

B. ¿Qué es?

Node.js es un entorno de ejecución (runtime) de JavaScript que permite correr ese lenguaje fuera del navegador: en tu computadora o en un servidor. Gracias a Node.js, el código del backend recibe solicitudes, valida datos, ejecuta reglas de negocio y devuelve respuestas.

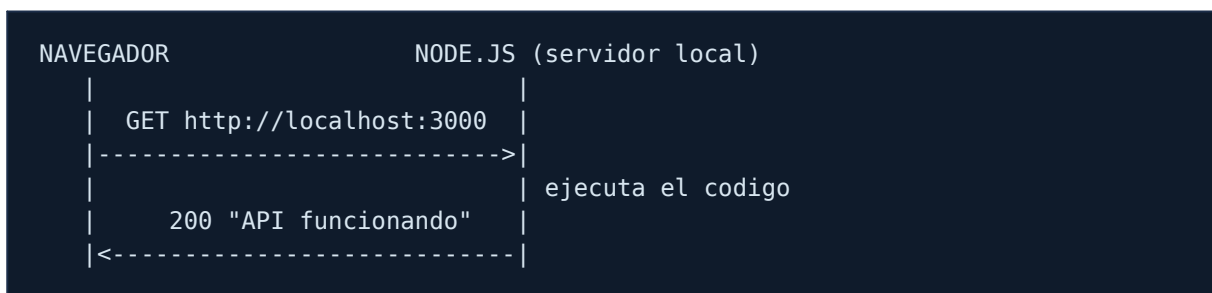
C. ¿Por qué existe?

El navegador ejecuta JavaScript pensado para interfaces: botones, formularios, el DOM. Pero un servidor necesita hacer otras cosas: escuchar peticiones de red, leer archivos, hablar con una base de datos. Node.js permite usar el mismo lenguaje (JavaScript) también para eso.

D. ¿Cómo funciona? (paso a paso)

- 1 Node.js ejecuta un archivo .js directamente en tu computadora, sin necesidad de un navegador.
- 2 Puede crear un servidor que "escucha" en un puerto específico de tu máquina.
- 3 Cuando llega una solicitud a ese puerto, ejecuta el código correspondiente.
- 4 Devuelve una respuesta al cliente que hizo la solicitud.

E. Diagrama visual



F. Ejemplo cotidiano (analogía)

El navegador es el cliente del restaurante; Node.js te permite construir la cocina y el personal que recibe y procesa los pedidos, usando el mismo idioma (JavaScript) que ya conoces del frontend.

G. Ejemplo dentro del proyecto EventHub

Crearás el primer servidor de EventHub: un programa Node.js que responde "API funcionando" cuando visitas `http://localhost:3000`, el primer paso antes de construir la API real.

H. Código mínimo

JAVASCRIPT

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.end('API funcionando');
});

server.listen(3000);
```

I. Explicación línea por línea

<code>const http = require('http');</code>	Importa el módulo incorporado de Node.js para crear servidores HTTP.
<code>const server = http.createServer((req, res) => {</code>	Crea un servidor; esta función se ejecuta cada vez que llega una solicitud (req) y arma una respuesta (res).
<code> res.writeHead(200, {'Content-Type': 'text/plain'});</code>	Indica que la respuesta será exitosa (200) y que el contenido es texto plano.
<code> res.end('API funcionando');</code>	Envía el texto "API funcionando" como cuerpo de la respuesta y la cierra.
<code>server.listen(3000);</code>	Pone al servidor a escuchar solicitudes en el puerto 3000 de tu computadora.

J. ¿Qué debería ver en pantalla?

Al ejecutar ``node servidor.js`` en la terminal y abrir `http://localhost:3000` en el navegador, verás el texto "API funcionando" en pantalla.

K. Error frecuente

⚠ Pensar que Node.js es un framework como Express; en realidad Node.js es el entorno que permite ejecutar JavaScript en el servidor, y Express es una herramienta que se construye sobre Node.js para facilitar el trabajo.

L. Cómo encontrar el error

Si ``node servidor.js`` da un error de puerto ocupado ("EADDRINUSE"), significa que ya hay otro programa usando el puerto 3000; ciérralo o usa otro puerto.

M. Ejercicio guiado

Crea un servidor local que responda "API funcionando" en `http://localhost:3000`, ejecútalo con ``node`` y confirma el resultado en el navegador.

N. Ejercicio independiente

Modifica el servidor para que responda un mensaje distinto según la ruta solicitada (por ejemplo, ``/`` y ``/eventos`` devuelven textos diferentes), sin ver ningún ejemplo similar.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Qué diferencia hay entre ejecutar JavaScript en el navegador y ejecutarlo con Node.js?

Respuesta de referencia: En el navegador, JavaScript trabaja con la interfaz y APIs del cliente; con Node.js puede ejecutarse en el servidor y acceder a servicios del sistema como archivos o red.

[COMPRENSIÓN] ¿Qué hace ``server.listen(3000)`` ?

Respuesta de referencia: Hace que el servidor empiece a escuchar solicitudes entrantes en el puerto 3000 de la computadora.

[APLICACIÓN] ¿Qué comando de terminal usarías para ejecutar un archivo llamado `servidor.js` con Node?

Respuesta de referencia: `node servidor.js`

[DIAGNÓSTICO] El navegador muestra "No se puede acceder a este sitio" al visitar `localhost:3000`. ¿Qué revisarías primero?

Respuesta de referencia: Si el servidor sigue corriendo en la terminal (si no hubo un error que lo detuvo) y si el puerto coincide con el que escribiste en la URL.

[TRANSFERENCIA] ¿Cómo harías que el servidor responda con JSON en vez de texto plano?

Respuesta de referencia: Cambiando el Content-Type a 'application/json' y usando `JSON.stringify()` sobre un objeto en `res.end()`.

P. Mini reto

Crea un servidor Node.js que responda con la hora actual del servidor cada vez que se le hace una solicitud, sin ver ningún ejemplo del curso.

Q. Checkpoint

✓ **¿Puedes explicar de memoria qué es un runtime y por qué Node.js permite usar JavaScript fuera del navegador?**

MÓDULO 11

Express y REST API

A. Lo que vas a aprender

Vas a construir endpoints de una API de forma organizada. Al terminar tendrás una API de eventos completa con las cuatro operaciones básicas.

B. ¿Qué es?

Express es una herramienta (framework) que facilita crear rutas y middleware en Node.js, en vez de escribir todo a mano como en el módulo anterior. Una API REST usa recursos (como "eventos") y métodos HTTP para expresar operaciones comunes. CRUD significa Crear, leer (Read), actualizar (Update) y eliminar (Delete): las cuatro operaciones más comunes sobre cualquier recurso.

C. ¿Por qué existe?

Escribir un servidor a mano para cada ruta, con validaciones y manejo de errores, se vuelve repetitivo y propenso a errores. Express organiza ese trabajo con una sintaxis clara para definir rutas, y con middleware: funciones que se ejecutan antes de llegar a la lógica final (por ejemplo, para leer el cuerpo de la solicitud).

D. ¿Cómo funciona? (paso a paso)

- 1 Se crea una aplicación Express (`express()`).
- 2 Se definen rutas: un método HTTP, una dirección y una función que responde.
- 3 El middleware (como `express.json()`) procesa la solicitud antes de que llegue a la ruta.
- 4 Cada ruta responde con datos (normalmente JSON) o con un código de error si algo falla.

E. Diagrama visual

```
NAVEGADOR
|
| GET /eventos
v
EXPRESS ---busca la ruta que coincide--> app.get('/eventos', funcion)
|
v
FUNCION se ejecuta
|
v
res.json([...]) ---> JSON de vuelta al navegador
```

El diagrama ilustra el flujo de una solicitud en Express. Comienza con el navegador enviando una solicitud GET a la ruta /eventos. Esta solicitud es recibida por Express, que busca la ruta correspondiente (app.get('/eventos', funcion)). Una vez encontrada, se ejecuta la función asociada. Finalmente, la función devuelve una respuesta en formato JSON (res.json([...])) que es enviada de vuelta al navegador.

F. Ejemplo cotidiano (analogía)

Las rutas de Express son como mostradores especializados: cada uno recibe un tipo específico de solicitud ("quiero ver la lista", "quiero crear uno nuevo") y entrega una respuesta siguiendo reglas claras.

G. Ejemplo dentro del proyecto EventHub

Construirás la API real de EventHub con Express: listar eventos, ver uno por id, crear, actualizar y eliminar, devolviendo errores claros cuando el recurso no existe.

H. Código mínimo

JAVASCRIPT

```
const express = require('express');
const app = express();
app.use(express.json());

app.get('/eventos', (req, res) => {
  res.json([ { id: 1, nombre: 'Festival' } ]);
});

app.listen(3000);
```

I. Explicación línea por línea

<code>const express = require('express');</code>	Importa la librería Express.
<code>const app = express();</code>	Crea la aplicación principal del servidor.
<code>app.use(express.json());</code>	Middleware: permite que Express entienda cuerpos de solicitud en formato JSON.
<code>app.get('/eventos', (req, res) => {</code>	Define una ruta: cuando llega un GET a /eventos, ejecuta esta función.
<code>res.json([{ id: 1, nombre: 'Festival' }]]);</code>	Responde con un array de eventos en formato JSON.
<code>app.listen(3000);</code>	Pone al servidor a escuchar en el puerto 3000.

J. ¿Qué debería ver en pantalla?

Al visitar `http://localhost:3000/eventos` en el navegador (o con una herramienta como Postman/curl), verás un JSON con la lista de eventos, por ejemplo ``[{"id":1,"nombre":"Festival"}]``.

K. Error frecuente

⚠ Aceptar cualquier `body` enviado por el cliente sin validarlo, lo que permite que datos incompletos o incorrectos lleguen hasta la base de datos o rompan la lógica del servidor.

L. Cómo encontrar el error

Agrega `console.log(req.body)` al inicio de cada ruta que reciba datos, y confirma que llega exactamente lo que esperabas antes de usarlo.

M. Ejercicio guiado

Implementa `/eventos` y luego agrega GET por id, POST, PUT y DELETE. Devuelve errores claros (con el código de estado adecuado) cuando el recurso no existe.

N. Ejercicio independiente

Agrega validación: si el POST no incluye un campo "nombre", responde con un error 400 y un mensaje claro, sin ver ningún ejemplo similar.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Por qué una API no debe asumir que todos los datos recibidos son correctos?

Respuesta de referencia: Porque el cliente puede enviar datos incompletos, incorrectos o maliciosos, y el servidor debe proteger sus reglas y datos validando siempre.

[COMPRENSIÓN] ¿Qué hace `express.json()` como middleware?

Respuesta de referencia: Permite que Express lea y entienda el cuerpo de una solicitud cuando viene en formato JSON, y lo pone disponible en `req.body`.

[APLICACIÓN] ¿Cómo definirías una ruta que responda a `GET /eventos/:id`?

Respuesta de referencia: `app.get('/eventos/:id', (req, res) => { ... })`, usando `req.params.id` para leer el id.

[DIAGNÓSTICO] El endpoint POST `/eventos` siempre recibe `req.body` como `undefined`. ¿Qué revisarías primero?

Respuesta de referencia: Si `app.use(express.json())` está presente antes de definir la ruta.

[TRANSFERENCIA] ¿Cómo devolverías un error 404 si alguien pide un evento con un id que no existe?

Respuesta de referencia: Buscando el evento por id, y si no se encuentra, usando `res.status(404).json({ error: 'No encontrado' })`.

P. Mini reto

Agrega un endpoint `GET /eventos/ciudad/:ciudad` que devuelva solo los eventos de esa ciudad, sin ver ningún ejemplo del curso.

Q. Checkpoint

✓ ¿Puedes explicar de memoria qué es una ruta, qué es un middleware y qué significa CRUD?

MÓDULO 12

SQL y PostgreSQL

A. Lo que vas a aprender

Vas a aprender a almacenar y consultar datos de forma estructurada. Al terminar podrás crear tablas relacionadas y hacer consultas básicas.

B. ¿Qué es?

Una base de datos relacional organiza la información en tablas relacionadas entre sí. SQL (Structured Query Language) es el lenguaje para consultar y modificar esos datos. PostgreSQL es un sistema de gestión de bases de datos relacional, uno de los más usados en el mundo real. Una tabla tiene filas (registros) y columnas (campos). La clave primaria (primary key) identifica de forma única cada fila; una clave foránea (foreign key) conecta una tabla con otra.

C. ¿Por qué existe?

Guardar datos en variables o archivos sueltos funciona para pruebas pequeñas, pero no resiste miles de registros, múltiples usuarios accediendo a la vez, ni relaciones complejas entre datos. Las bases de datos relacionales están diseñadas justamente para eso: reglas, relaciones, concurrencia y consultas a mayor escala.

D. ¿Cómo funciona? (paso a paso)

- 1 Se define una tabla con columnas y sus tipos de dato (texto, número, fecha...).
- 2 Cada fila nueva se guarda de forma permanente en la base de datos.
- 3 SQL permite pedir (SELECT), insertar (INSERT), modificar (UPDATE) o borrar (DELETE) esas filas.
- 4 Las relaciones entre tablas (por ejemplo, eventos y usuarios) se hacen con claves foráneas.

E. Diagrama visual

TABLA: eventos

```
+---+-----+-----+-----+
| id | nombre  | fecha    | ciudad |
+---+-----+-----+-----+
| 1  | Festival | 2026-08-25 | CDMX   |
| 2  | Concierto | 2026-09-10 | GDL    |
+---+-----+-----+-----+
```

^

| clave primaria (identifica cada fila de forma unica)

F. Ejemplo cotidiano (analogía)

Una hoja de cálculo bien estructurada se parece un poco a una tabla de base de datos. La diferencia es que una base de datos está diseñada para reglas, relaciones, muchas personas usándola al mismo tiempo, seguridad y consultas a mayor escala.

G. Ejemplo dentro del proyecto EventHub

Crearás las tablas `usuarios`, `eventos` e `inscripciones` de EventHub, con una relación entre usuario y evento a través de la tabla de inscripciones.

H. Código mínimo

SQL

```
CREATE TABLE eventos (  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  fecha DATE NOT NULL,  
  ciudad VARCHAR(100)  
);  
  
SELECT * FROM eventos;
```

I. Explicación línea por línea

CREATE TABLE eventos (	Crea una nueva tabla llamada eventos.
id SERIAL PRIMARY KEY,	Columna id: un número que se genera automáticamente y sirve como clave primaria.
nombre VARCHAR(100) NOT NULL,	Columna nombre: texto de hasta 100 caracteres, obligatorio (no puede quedar vacío).
fecha DATE NOT NULL,	Columna fecha: un valor de tipo fecha, también obligatorio.
ciudad VARCHAR(100)	Columna ciudad: texto opcional.
SELECT * FROM eventos;	Consulta que devuelve todas las columnas de todas las filas de la tabla eventos.

J. ¿Qué debería ver en pantalla?

Después de crear la tabla e insertar algún dato de prueba, `SELECT \* FROM eventos;` mostrará una tabla de resultados con las columnas id, nombre, fecha y ciudad.

K. Error frecuente

⚠ Crear tablas sin relaciones claras entre ellas o sin claves definidas, lo que dificulta (o hace imposible) saber qué evento pertenece a qué usuario más adelante.

L. Cómo encontrar el error

Antes de crear las tablas, dibuja en papel qué tablas necesitas y qué relaciones hay entre ellas; si no puedes dibujar una flecha clara entre dos tablas, probablemente falta una clave foránea.

M. Ejercicio guiado

Crea las tablas usuarios, eventos e inscripciones. Agrega una relación entre usuario y evento usando claves foráneas en la tabla de inscripciones.

N. Ejercicio independiente

Escribe una consulta SQL que devuelva solo los eventos de la ciudad "CDMX", ordenados por fecha, sin ver ningún ejemplo similar.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Por qué cada tabla necesita una clave primaria?

Respuesta de referencia: Porque identifica de forma única cada fila y sirve como referencia para relacionarla con otras tablas.

[COMPRENSIÓN] ¿Qué es una clave foránea?

Respuesta de referencia: Una columna que referencia la clave primaria de otra tabla, creando una relación entre ambas.

[APLICACIÓN] ¿Qué instrucción SQL usarías para insertar un nuevo evento?

Respuesta de referencia: INSERT INTO eventos (nombre, fecha, ciudad) VALUES ('Feria', '2026-10-01', 'MTY');

[DIAGNÓSTICO] Al intentar insertar un evento sin fecha, la base de datos rechaza la operación. ¿Por qué?

Respuesta de referencia: Porque la columna fecha se definió como NOT NULL, es decir, obligatoria.

[TRANSFERENCIA] ¿Cómo diseñarías la relación para que un evento pueda tener varios organizadores?

Respuesta de referencia: Con una tabla intermedia (por ejemplo, evento_organizadores) que relacione eventos y usuarios mediante claves foráneas, similar a inscripciones.

P. Mini reto

Diseña y crea la tabla `comentarios`, relacionada con `eventos` y `usuarios` mediante claves foráneas, sin ver ningún ejemplo del curso.

Q. Checkpoint

✓ ¿Puedes dibujar de memoria las tablas usuarios, eventos e inscripciones con sus relaciones?

MÓDULO 13

Persistencia: Express + PostgreSQL

A. Lo que vas a aprender

Vas a conectar tu API con datos permanentes. Al terminar, los datos de EventHub sobrevivirán aunque reinicies el servidor.

B. ¿Qué es?

Hasta ahora el servidor puede responder, pero los datos escritos directamente en el código desaparecen al reiniciarlo. Aquí conectamos Express con PostgreSQL para que las rutas lean y escriban datos reales y permanentes. Una consulta parametrizada es una forma segura de insertar valores dentro de una consulta SQL sin concatenar texto directamente.

C. ¿Por qué existe?

Una aplicación real necesita que los datos sobrevivan reinicios, fallos y el paso del tiempo. Guardar todo en variables de JavaScript es útil para practicar, pero no sirve para un producto real; PostgreSQL resuelve ese problema de forma robusta.

D. ¿Cómo funciona? (paso a paso)

- 1 El servidor abre una conexión (o un pool de conexiones) hacia la base de datos.
- 2 Cuando llega una solicitud, la ruta ejecuta una consulta SQL usando esa conexión.
- 3 Los valores que vienen del usuario se pasan como parámetros, nunca pegados directamente al texto SQL.
- 4 El resultado de la consulta se transforma en la respuesta que recibe el cliente.

E. Diagrama visual

```
NAVEGADOR --GET /eventos/15--> EXPRESS --query SQL--> POSTGRESQL
                                         |
NAVEGADOR <----- JSON <----- EXPRESS <---- filas -----+
```

F. Ejemplo cotidiano (analogía)

El backend es el encargado de tomar el pedido; PostgreSQL es el registro permanente donde se conserva la información, como un archivero que no se borra aunque apagues las luces de la oficina.

G. Ejemplo dentro del proyecto EventHub

Harás que `POST /eventos` guarde un evento real en PostgreSQL y que `GET /eventos` lo recupere desde ahí, reemplazando los datos escritos a mano en el código.

H. Código mínimo

```
const result = await pool.query(
  'SELECT * FROM eventos WHERE id = $1',
  [eventId]
);
res.json(result.rows[0]);
```

I. Explicación línea por línea

<code>const result = await pool.query(</code>	Ejecuta una consulta SQL usando el pool de conexiones a PostgreSQL, y espera el resultado.
<code>'SELECT * FROM eventos WHERE id = \$1',</code>	La consulta SQL, con \$1 como marcador de posición para un valor seguro.
<code>[eventId]</code>	El valor real que reemplaza a \$1; PostgreSQL lo trata siempre como dato, nunca como código SQL.
<code>res.json(result.rows[0]);</code>	Responde con la primera fila encontrada, convertida a JSON.

J. ¿Qué debería ver en pantalla?

Al hacer `POST /eventos` con un nuevo evento y luego `GET /eventos`, el nuevo evento aparece en la lista, incluso si reinicias el servidor entre ambas peticiones.

K. Error frecuente

⚠ Construir SQL concatenando texto directamente con datos del usuario (por ejemplo `'SELECT * FROM eventos WHERE id = ' + eventId``), lo que abre la puerta a inyección SQL.

L. Cómo encontrar el error

Busca en tu código cualquier consulta SQL armada con ``+`` o plantillas de texto que incluyan datos del usuario directamente; reemplázala por una consulta parametrizada con ``$1``, ``$2``, etc.

M. Ejercicio guiado

Haz que `POST /eventos` guarde un evento real y que `GET /eventos` lo recupere desde PostgreSQL, usando siempre consultas parametrizadas.

N. Ejercicio independiente

Implementa `PUT /eventos/:id` y `DELETE /eventos/:id` conectados a PostgreSQL, sin ver ningún ejemplo similar.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Por qué conviene usar consultas parametrizadas?

Respuesta de referencia: Porque separan los datos enviados por el usuario de la instrucción SQL y ayudan a prevenir inyección SQL.

[COMPRENSIÓN] ¿Qué representa `result.rows` después de una consulta `SELECT`?

Respuesta de referencia: Un array con las filas que devolvió la consulta, cada una como un objeto.

[APLICACIÓN] ¿Cómo escribirías una consulta parametrizada para buscar eventos por ciudad?

Respuesta de referencia: `pool.query('SELECT * FROM eventos WHERE ciudad = $1', [ciudad])`

[DIAGNÓSTICO] Tu API guarda eventos correctamente, pero al reiniciar el servidor todo desaparece. ¿Qué indicaría eso?

Respuesta de referencia: Que los datos se están guardando en una variable de JavaScript en memoria, no realmente en PostgreSQL.

[TRANSFERENCIA] ¿Cómo adaptarías la consulta de ejemplo para buscar un evento por nombre exacto en vez de por id?

Respuesta de referencia: `pool.query('SELECT * FROM eventos WHERE nombre = $1', [nombre])`

P. Mini reto

Conecta el endpoint `DELETE /eventos/:id` a PostgreSQL para que borre realmente la fila correspondiente, sin ver ningún ejemplo del curso.

Q. Checkpoint

✓ **¿Puedes explicar de memoria por qué nunca debes concatenar datos del usuario directamente dentro de una consulta SQL?**

MÓDULO 14

Autenticación y autorización

A. Lo que vas a aprender

Vas a distinguir identidad de permisos. Al terminar podrás proteger una ruta exclusiva para administradores.

B. ¿Qué es?

Autenticación responde "¿quién eres?". Autorización responde "¿qué puedes hacer?". Nunca se deben guardar contraseñas en texto plano: deben almacenarse con funciones de hash adecuadas. Un token o una sesión es la forma en que el servidor recuerda que ya verificó tu identidad en solicitudes futuras.

C. ¿Por qué existe?

Muchas aplicaciones necesitan saber quién hace cada solicitud (para mostrar datos personalizados, proteger información) y qué le está permitido hacer (un usuario normal no debería poder borrar eventos ajenos). Separar estos dos conceptos evita errores graves de seguridad.

D. ¿Cómo funciona? (paso a paso)

- 1 El usuario se registra: su contraseña se transforma con hash antes de guardarse.
- 2 El usuario inicia sesión: el servidor compara el hash de la contraseña enviada con el guardado.
- 3 Si coincide, el servidor entrega una prueba de identidad (sesión o token) al cliente.
- 4 En solicitudes futuras, el cliente envía esa prueba, y el servidor decide qué le permite hacer según su rol.

E. Diagrama visual

```
REGISTRO
  |
  v
USUARIO (contraseña guardada como hash)
  |
  v
LOGIN --credenciales correctas--> SESION / TOKEN
  |
  v
RECURSOS PROTEGIDOS (segun el rol del usuario)
```

F. Ejemplo cotidiano (analogía)

Al entrar a un edificio, primero verifican tu identidad (autenticación) en la recepción. Después, una tarjeta de acceso decide a qué habitaciones puedes entrar (autorización): estar identificado no significa tener acceso a todo.

G. Ejemplo dentro del proyecto EventHub

Implementarás el registro e inicio de sesión de EventHub, y protegerás la ruta para crear eventos para que solo usuarios con rol "organizador" puedan usarla.

H. Código mínimo

JAVASCRIPT

```
const user = { id: 7, role: 'admin' };

if (user.role !== 'admin') {
  return res.status(403).json({ error: 'Prohibido' });
}
```

I. Explicación línea por línea

<code>const user = { id: 7, role: 'admin' };</code>	Representa al usuario que hizo la solicitud (normalmente vendría de la sesión o el token).
<code>if (user.role !== 'admin') {</code>	Verifica si el rol del usuario NO es admin.
<code> return res.status(403).json({ error: 'Prohibido' });</code>	Si no es admin, responde con el código 403 (prohibido) y detiene la ejecución de la ruta.

J. ¿Qué debería ver en pantalla?

Un usuario con rol "admin" puede acceder a la ruta protegida con normalidad; un usuario con otro rol recibe una respuesta con código 403 y el mensaje "Prohibido".

K. Error frecuente

⚠ Confundir estar autenticado con tener permisos de administrador, es decir, asumir que cualquier usuario que inició sesión puede hacer cualquier cosa.

L. Cómo encontrar el error

Antes de cada acción sensible (crear, editar, borrar), pregúntate: ¿esta ruta verifica solo que hay una sesión, o también verifica el rol correcto?

M. Ejercicio guiado

Implementa registro e inicio de sesión y protege una ruta exclusiva para administradores, devolviendo 403 si el usuario no tiene el rol correcto.

N. Ejercicio independiente

Agrega un tercer rol ("asistente") que solo pueda inscribirse a eventos pero no crear ni editar, sin ver ningún ejemplo similar.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Puede un usuario autenticado hacer cualquier cosa? Explica por qué no.

Respuesta de referencia: No. Estar autenticado solo demuestra identidad; los permisos se deciden mediante autorización y roles o reglas específicas.

[COMPRENSIÓN] ¿Por qué nunca se debe guardar una contraseña en texto plano?

Respuesta de referencia: Porque si la base de datos se filtra, cualquiera con acceso vería las contraseñas reales de todos los usuarios; el hash protege ese dato incluso en ese escenario.

[APLICACIÓN] ¿Qué código de estado HTTP es apropiado cuando alguien sin permisos intenta hacer algo prohibido?

Respuesta de referencia: 403 (Forbidden).

[DIAGNÓSTICO] Un usuario reporta que puede ver el botón "Eliminar evento" aunque no es organizador, y al usarlo, funciona. ¿Qué falta?

Respuesta de referencia: Falta la verificación de autorización en el backend; mostrar u ocultar el botón en el frontend no es suficiente, la ruta también debe validar el rol.

[TRANSFERENCIA] ¿Cómo protegerías una ruta para que solo el usuario dueño de un evento (no cualquier organizador) pueda editarlo?

Respuesta de referencia: Comparando el id del usuario autenticado con el id del organizador guardado en el evento, además de verificar el rol.

P. Mini reto

Diseña, sin ver ningún ejemplo, la lógica de autorización para que un usuario solo pueda cancelar sus propias inscripciones, no las de otros usuarios.

Q. Checkpoint

✓ **¿Puedes explicar de memoria la diferencia entre autenticación y autorización con un ejemplo distinto al del edificio?**

MÓDULO 15

Seguridad web básica

A. Lo que vas a aprender

Vas a reconocer riesgos comunes y aprender a reducir su impacto. Al terminar podrás identificar y corregir vulnerabilidades típicas en una API.

B. ¿Qué es?

La seguridad no es un módulo que se agrega al final: es una forma de pensar durante todo el desarrollo. Implica validar entradas, limitar permisos, proteger secretos (contraseñas, claves) y nunca confiar ciegamente en datos enviados por el cliente.

C. ¿Por qué existe?

Cualquier aplicación conectada a internet puede recibir solicitudes maliciosas, no solo de usuarios legítimos. Sin prácticas básicas de seguridad, datos sensibles pueden filtrarse, modificarse o borrarse por personas no autorizadas.

D. ¿Cómo funciona? (paso a paso)

- 1 Cada amenaza tiene un patrón: alguien envía datos manipulados esperando que el sistema los use sin revisar.
- 2 La defensa suele consistir en validar, escapar o parametrizar esos datos antes de usarlos.
- 3 Los secretos (contraseñas de base de datos, claves de API) se guardan fuera del código, en variables de entorno.
- 4 CORS controla qué otros sitios pueden hacer solicitudes a tu API desde el navegador.

E. Diagrama visual

AMENAZA	QUE HACE	DEFENSA
-----	-----	-----
SQL Injection	inserta SQL malicioso en un campo	consultas parametrizadas
XSS	inserta scripts en el contenido	escapar/validar HTML
Secretos en Git	claves quedan expuestas	variables de entorno (.env)
CORS mal configurado	cualquier sitio llama tu API	restringir orígenes permitidos

F. Ejemplo cotidiano (analogía)

Una puerta con buena cerradura sigue siendo insegura si dejas las llaves pegadas en la puerta. La seguridad implica pensar tanto en el acceso (la cerradura) como en el comportamiento posterior (dónde dejas las llaves).

G. Ejemplo dentro del proyecto EventHub

Auditarás la API de EventHub buscando específicamente: campos sin validar, contraseñas mal guardadas, secretos escritos directamente en el código y consultas SQL sin parametrizar.

H. Código mínimo

JAVASCRIPT

```
// Nunca confíes en el dato enviado por el cliente.  
const sql = 'SELECT * FROM usuarios WHERE email = $1';  
await pool.query(sql, [email]);
```

I. Explicación línea por línea

// Nunca confíes en el dato enviado por el cliente.	Comentario recordatorio: `email` viene del usuario y podría contener cualquier cosa.
const sql = 'SELECT * FROM usuarios WHERE email = \$1';	La consulta usa un marcador de posición (\$1) en vez de insertar el email directamente en el texto.
await pool.query(sql, [email]);	PostgreSQL trata el valor de email siempre como dato, nunca como parte del comando SQL, evitando inyección.

J. ¿Qué debería ver en pantalla?

Al probar con un email que incluya comillas o código SQL malicioso, la consulta parametrizada lo trata como texto normal, sin romper la base de datos ni devolver datos que no debería.

K. Error frecuente

⚠ Guardar contraseñas, claves de base de datos o tokens de API directamente en el código y subirlo a Git, dejándolos visibles para cualquiera que vea el repositorio.

L. Cómo encontrar el error

Busca en tu código cualquier contraseña o clave escrita directamente entre comillas; si la encuentras, muévela a un archivo `.env` que no se suba a Git.

M. Ejercicio guiado

Encuentra y corrige tres vulnerabilidades intencionales en una API de práctica: una de inyección SQL, una de secretos expuestos y una de validación faltante.

N. Ejercicio independiente

Revisa tu propia API de EventHub y documenta, en tu README, qué validaciones de seguridad ya tiene y cuáles le faltan.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Por qué nunca debes enviar secretos como contraseñas o claves privadas dentro del repositorio?

Respuesta de referencia: Porque el repositorio puede compartirse o exponerse; un secreto comprometido puede permitir acceso no autorizado a servicios.

[COMPRENSIÓN] ¿Qué es XSS en términos simples?

Respuesta de referencia: Un ataque donde alguien logra insertar código (normalmente JavaScript) dentro del contenido de tu página, que se ejecuta cuando otros usuarios la visitan.

[APLICACIÓN] ¿Dónde deberías guardar la contraseña de conexión a tu base de datos?

Respuesta de referencia: En una variable de entorno, típicamente en un archivo .env que no se sube a Git.

[DIAGNÓSTICO] Un formulario de comentarios permite que el texto ingresado se muestre tal cual en la página de otros usuarios, incluyendo etiquetas HTML. ¿Qué riesgo hay?

Respuesta de referencia: Riesgo de XSS: alguien podría insertar un script malicioso en un comentario que se ejecute en el navegador de otros usuarios.

[TRANSFERENCIA] ¿Cómo protegerías un endpoint que permite subir imágenes de que alguien suba archivos peligrosos?

Respuesta de referencia: Validando el tipo real de archivo, limitando el tamaño, y nunca ejecutando el archivo subido como código.

P. Mini reto

Revisa un formulario que hayas construido en módulos anteriores y lista, sin ayuda, tres formas en que alguien podría intentar abusar de él.

Q. Checkpoint

✓ **¿Puedes explicar de memoria qué es SQL injection y cómo se previene?**

MÓDULO 16

Testing y debugging

A. Lo que vas a aprender

Vas a aprender a encontrar errores de manera sistemática y a demostrar que tu software funciona. Al terminar podrás escribir tu primera prueba automatizada.

B. ¿Qué es?

Debugging es investigar por qué el programa no se comporta como esperabas. Testing es comprobar de manera repetible que ciertas reglas siguen funcionando, incluso después de cambiar el código. Los dos son habilidades normales del desarrollo, no un castigo posterior a programar.

C. ¿Por qué existe?

El código cambia constantemente, y sin pruebas, es fácil romper algo que funcionaba sin darte cuenta (una regresión). El debugging sistemático evita perder horas "adivinando" la causa de un error.

D. ¿Cómo funciona? (paso a paso)

- 1 Debugging: reproducir el error, observar evidencia (consola, Network), aislar la causa, probar una hipótesis, corregir y verificar.
- 2 Testing: escribir una entrada conocida, ejecutar la función, y comprobar si el resultado es el esperado.
- 3 Un test automatizado se puede volver a correr en segundos cada vez que cambias el código.

E. Diagrama visual

```
TESTING:
  ENTRADA --> FUNCION --> RESULTADO --> ¿ES LO ESPERADO?

DEBUGGING:
  ¿Que esperaba?
  |
  ¿Que ocurrio?
  |
  ¿Donde cambio el comportamiento?
  |
  ¿Que hipotesis tengo?
  |
  ¿Como la pruebo?
  |
  ¿Funciono?
```

F. Ejemplo cotidiano (analogía)

Debugging es trabajo detectivesco: buscas pistas y sospechosos hasta encontrar al culpable. Testing es como una lista de chequeo que repites cada vez que algo cambia, para confirmar que nada se rompió.

G. Ejemplo dentro del proyecto EventHub

Escribirás una prueba automatizada para la función `priceForAge` de EventHub, y luego romperás deliberadamente una ruta de la API para practicar debugging.

H. Código mínimo

JAVASCRIPT

```
test('suma precios', () => {  
  expect(calculateTotal(100, 3)).toBe(300);  
});
```

I. Explicación línea por línea

<code>test('suma precios', () => {</code>	Define una prueba con un nombre descriptivo de lo que verifica.
<code> expect(calculateTotal(100, 3)).toBe(300);</code>	Ejecuta la función con datos conocidos y compara el resultado con el valor esperado (300).
<code>});</code>	Cierra la prueba.

J. ¿Qué debería ver en pantalla?

Al ejecutar el corredor de pruebas (por ejemplo, `npm test`), verás un reporte que indica si la prueba "suma precios" pasó (verde) o falló (rojo), con detalles si falló.

K. Error frecuente

⚠ Cambiar código intentando distintas cosas al azar sin antes reproducir el error de forma consistente, lo que hace muy difícil saber si una "solución" realmente funcionó o fue coincidencia.

L. Cómo encontrar el error

Antes de cambiar nada, confirma que puedes hacer que el error ocurra de nuevo, de forma predecible, con los mismos pasos exactos.

M. Ejercicio guiado

Rompe una ruta deliberadamente (por ejemplo, cambia un nombre de campo). Encuentra el error usando logs y la pestaña Network antes de revisar la solución.

N. Ejercicio independiente

Escribe una prueba automatizada para la función de descuentos por edad del Módulo 4, cubriendo al menos tres casos distintos, sin ver ningún ejemplo similar.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Qué diferencia hay entre corregir un bug y demostrar con una prueba que no volverá a aparecer?

Respuesta de referencia: Corregir el bug resuelve el problema actual; una prueba automatizada permite detectar si el mismo comportamiento vuelve a romperse en el futuro.

[COMPRENSIÓN] ¿Qué es una regresión?

Respuesta de referencia: Un error que reaparece porque un cambio nuevo rompió algo que antes funcionaba correctamente.

[APLICACIÓN] ¿Qué escribirías primero al notar un error: una corrección o una forma de reproducirlo?

Respuesta de referencia: Primero una forma confiable de reproducirlo; sin eso, no puedes confirmar si la corrección realmente funcionó.

[DIAGNÓSTICO] Una prueba pasa en tu computadora pero falla en la de un compañero. ¿Qué hipótesis investigarías primero?

Respuesta de referencia: Diferencias de entorno: versión de Node, datos de prueba distintos, o dependencias instaladas de forma diferente.

[TRANSFERENCIA] ¿Cómo escribirías una prueba para el endpoint GET /eventos/:id cuando el id no existe?

Respuesta de referencia: Simulando una solicitud con un id inexistente y comprobando que la respuesta tiene el código de estado 404, no un error genérico.

P. Mini reto

Escribe dos pruebas automatizadas más para `priceForAge`: un caso límite exacto en 12 años y otro en 18 años, sin ver ningún ejemplo del curso.

Q. Checkpoint

✓ **¿Puedes explicar de memoria los pasos del proceso de debugging sistemático, en orden?**

MÓDULO 17

Frontend moderno con React

A. Lo que vas a aprender

Vas a entender qué problema resuelve un framework como React y a construir componentes reutilizables. Al terminar podrás convertir una tarjeta de evento en un componente.

B. ¿Qué es?

Nuestro proyecto está creciendo. Tenemos tarjetas repetidas, formularios, listas que cambian según filtros: estados que cambian constantemente. React ayuda a construir interfaces a partir de componentes (piezas reutilizables de interfaz) y estado (los datos que pueden cambiar dentro de la aplicación).

C. ¿Por qué existe?

Manipular el DOM directamente, como en el Módulo 5, funciona bien para páginas pequeñas, pero se vuelve difícil de mantener cuando hay muchas piezas interconectadas que cambian. React organiza esa complejidad: describes cómo se ve la interfaz según los datos actuales, y React se encarga de actualizar el DOM cuando cambió.

D. ¿Cómo funciona? (paso a paso)

- 1 Divides la interfaz en componentes: piezas reutilizables con su propia lógica y apariencia.
- 2 Cada componente puede recibir datos desde afuera (props) y tener sus propios datos internos que cambian (state).
- 3 Cuando el estado cambia, React vuelve a calcular qué debe mostrarse y actualiza solo lo necesario en el DOM.
- 4 Los hooks (como useState) son funciones especiales que dan estas capacidades a los componentes.

E. Diagrama visual



F. Ejemplo cotidiano (analogía)

Si tu interfaz fuera una ciudad, un componente es un edificio reutilizable (puedes construir el mismo diseño de edificio en varios lugares) y el estado son los datos que cambian dentro de esa ciudad, como el número de personas en cada edificio en un momento dado.

G. Ejemplo dentro del proyecto EventHub

Convertirás la lista de eventos de EventHub en componentes React: un `EventList` que recibe todos los eventos y un `EventCard` que recibe uno solo, con un estado para el filtro por ciudad.

H. Código mínimo

JSX

```
function EventCard({ event }) {  
  return (  
    <article>  
      <h2>{event.nombre}</h2>  
    </article>  
  );  
}
```

I. Explicación línea por línea

<code>function EventCard({ event })</code>	Define un componente que recibe un objeto de props, y de ahí extrae directamente `event`.
<code>{</code>	
<code>return (</code>	Un componente React devuelve lo que debe mostrarse en pantalla.
<code><article></code>	Se ve como HTML, pero es JSX: una forma de escribir la interfaz dentro de JavaScript.
<code><h2>{event.nombre}</h2></code>	Las llaves `{}` insertan un valor de JavaScript (en este caso, el nombre del evento) dentro del JSX.

J. ¿Qué debería ver en pantalla?

Al renderizar `` , verás una tarjeta con el título "Festival" en pantalla, igual que en el Módulo 2, pero ahora generado por un componente reutilizable.

K. Error frecuente

⚠ Aprender React sin comprender bien JavaScript primero, lo que hace muy difícil entender por qué las cosas funcionan (o no funcionan) dentro de los componentes.

L. Cómo encontrar el error

Si algo en React no tiene sentido, prueba primero a escribir la misma lógica en JavaScript puro (sin React); si ahí tampoco tiene sentido, el problema es de JavaScript, no de React.

M. Ejercicio guiado

Convierte la lista de eventos de EventHub en componentes React y agrega un estado (`useState`) para el filtro por ciudad.

N. Ejercicio independiente

Crea un componente `EventForm` que permita agregar un nuevo evento a la lista usando estado, sin ver ningún ejemplo similar.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Qué ventaja tiene un componente reutilizable frente a copiar el mismo HTML varias veces?

Respuesta de referencia: Permite construir una pieza de UI una vez y reutilizarla con datos diferentes, reduciendo duplicación y facilitando el mantenimiento.

[COMPRENSIÓN] ¿Qué diferencia hay entre props y state?

Respuesta de referencia: Las props son datos que un componente recibe desde afuera y no puede modificar directamente; el state son datos internos del componente que sí puede cambiar.

[APLICACIÓN] ¿Cómo pasarías un evento específico a un componente EventCard?

Respuesta de referencia: <EventCard event={miEvento} />

[DIAGNÓSTICO] Cambias el estado con `useState` pero la pantalla no se actualiza. ¿Qué revisarías primero?

Respuesta de referencia: Si estás usando la función que actualiza el estado (por ejemplo setEvents) en vez de modificar la variable directamente.

[TRANSFERENCIA] ¿Cómo estructurarías componentes para una página de detalle de evento con comentarios?

Respuesta de referencia: Un componente EventDetail que reciba el evento, y dentro, una lista de componentes Comment, cada uno recibiendo un comentario como prop.

P. Mini reto

Construye un componente `Counter` con un botón que aumente un número en pantalla usando useState, sin ver ningún ejemplo del curso.

Q. Checkpoint

✓ **¿Puedes explicar de memoria qué problema resuelve React frente a manipular el DOM directamente?**

MÓDULO 18

Calidad, accesibilidad y rendimiento

A. Lo que vas a aprender

Vas a hacer que la aplicación sea útil, inclusiva y razonablemente rápida. Al terminar podrás auditar tu propia app y corregir problemas reales.

B. ¿Qué es?

Una app no está terminada solo porque funciona en tu computadora. Debe ser usable, comprensible y resistente a errores. Accesibilidad implica que más personas puedan utilizarla (incluyendo quienes usan teclado o lectores de pantalla). Rendimiento significa reducir trabajo innecesario y tiempos de espera.

C. ¿Por qué existe?

Una interfaz que solo funciona con mouse, con buena vista y con internet rápido excluye a muchas personas reales. Ignorar accesibilidad y rendimiento no es un detalle estético: afecta quién puede usar tu aplicación y qué tan buena es la experiencia.

D. ¿Cómo funciona? (paso a paso)

- 1 Prueba tu app navegando solo con el teclado (Tab, Enter, flechas).
- 2 Revisa el contraste de colores entre texto y fondo.
- 3 Usa herramientas como Lighthouse para medir rendimiento y accesibilidad.
- 4 Reduce trabajo innecesario: imágenes muy pesadas, carga de datos que no se necesitan aún (lazy loading).

E. Diagrama visual

```
Primero: teclado (¿puedo llegar a todo sin mouse?)
|
Luego: contraste (¿se lee bien el texto?)
|
Luego: Network y rendimiento (¿que tan pesado es todo?)
|
Luego: vuelve a probar la experiencia completa
```

F. Ejemplo cotidiano (analogía)

Un edificio excelente no solo se ve bien: tiene rampas, señalización clara, salidas de emergencia y servicios que funcionan incluso cuando hay mucha gente. Una app de calidad funciona igual de bien en condiciones distintas a las ideales.

G. Ejemplo dentro del proyecto EventHub

Auditarás EventHub navegando solo con teclado, revisando contraste de colores, y usando DevTools para medir cuánto tarda en cargar la lista de eventos.

H. Código mínimo

TEXT

```
// Primero: teclado
// Luego: contraste
// Luego: Network y rendimiento
// Luego: vuelve a probar la experiencia completa
```

I. Explicación línea por línea

// Primero: teclado	Verifica que cada botón, enlace y campo se pueda alcanzar y activar solo con Tab y Enter.
// Luego: contraste	Confirma que el texto sea legible sobre su fondo, sin depender solo del color para transmitir información.
// Luego: Network y rendimiento	Revisa cuánto pesan las imágenes y cuánto tarda la página en volverse interactiva.
// Luego: vuelve a probar la experiencia completa	Después de corregir, repite todo el recorrido para confirmar que nada se rompió.

J. ¿Qué debería ver en pantalla?

Al usar solo el teclado, deberías poder llegar a cada botón y campo de EventHub, con un indicador visual claro de qué elemento está enfocado en cada momento.

K. Error frecuente

⚠ Medir la calidad de una app solo por su estética, olvidando por completo el teclado, el contraste y los tiempos de carga.

L. Cómo encontrar el error

Desconecta el mouse por cinco minutos y trata de completar una tarea común de tu app (crear un evento, por ejemplo) solo con el teclado.

M. Ejercicio guiado

Audita EventHub con teclado y DevTools. Corrige al menos cinco problemas de accesibilidad o rendimiento que encuentres.

N. Ejercicio independiente

Ejecuta Lighthouse sobre tu página y corrige las tres recomendaciones más importantes que aparezcan, sin ver ningún ejemplo similar.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Por qué accesibilidad forma parte de la calidad y no es un extra estético?

Respuesta de referencia: Porque una interfaz de calidad debe poder ser utilizada por personas con distintas capacidades y formas de interacción, no solo verse bien.

[COMPRENSIÓN] ¿Qué mide Lighthouse?

Respuesta de referencia: Aspectos como rendimiento, accesibilidad, buenas prácticas y SEO de una página web.

[APLICACIÓN] ¿Cómo comprobarías si un botón es alcanzable solo con teclado?

Respuesta de referencia: Navegando la página presionando solo Tab y Enter, sin tocar el mouse en ningún momento.

[DIAGNÓSTICO] Un usuario reporta que no puede saber en qué campo está escribiendo cuando navega con teclado. ¿Qué problema de accesibilidad es ese?

Respuesta de referencia: Falta de un indicador visual de foco (focus) claro en los elementos interactivos.

[TRANSFERENCIA] ¿Cómo mejorarías el rendimiento de una página que carga 50 imágenes de eventos a la vez?

Respuesta de referencia: Aplicando lazy loading, para que las imágenes fuera de la pantalla visible se carguen solo cuando el usuario se acerca a ellas.

P. Mini reto

Revisa tres páginas web que uses seguido navegando solo con teclado, y anota qué tan bien (o mal) funcionan, sin ayuda del curso.

Q. Checkpoint

✓ **¿Puedes explicar de memoria por qué el rendimiento y la accesibilidad son parte de la calidad de una app, no extras opcionales?**

MÓDULO 19

Deployment y proyecto de portafolio

A. Lo que vas a aprender

Vas a pasar de localhost a una aplicación accesible desde internet. Al terminar tendrás EventHub publicado con una URL real.

B. ¿Qué es?

Deployment significa publicar la aplicación en una infraestructura real, fuera de tu computadora. Aquí aparecen conceptos como build (preparar el código para producción), variables de entorno, dominio, HTTPS y logs de producción.

C. ¿Por qué existe?

Una aplicación que solo funciona en tu computadora no sirve de mucho para otras personas. El deployment es el paso que convierte tu proyecto de práctica en algo que cualquiera puede visitar desde un navegador, en cualquier parte del mundo.

D. ¿Cómo funciona? (paso a paso)

- 1 Subes tu código a GitHub.
- 2 Conectas ese repositorio a un servicio de hosting.
- 3 El servicio construye (build) tu aplicación y la ejecuta en un servidor real.
- 4 Configuras variables de entorno de producción (por ejemplo, la conexión a la base de datos).
- 5 El servicio te entrega una dirección (dominio) donde tu app queda disponible con HTTPS.

E. Diagrama visual

```
MI COMPUTADORA
  |
  v
GitHub
  |
  v
HOSTING --construye (build)--> SERVIDOR (produccion)
  |
  v
DOMINIO (https://...)
  |
  v
USUARIO (en cualquier parte del mundo)
```

F. Ejemplo cotidiano (analogía)

Desarrollar en tu casa es el ensayo; hacer deployment es abrir el negocio al público. Ya importan la configuración, la seguridad, los costos y el monitoreo, porque ahora hay personas reales usando la aplicación.

G. Ejemplo dentro del proyecto EventHub

Publicarás una versión funcional de EventHub, con su base de datos conectada, y escribirás un README que explique cómo ejecutarlo localmente y cómo desplegarlo.

H. Código mínimo

TEXT

```
# Produccion
DATABASE_URL=...
PORT=3000
NODE_ENV=production
```

I. Explicación línea por línea

DATABASE_URL=...	La dirección de conexión a la base de datos de producción, distinta de la que usas en tu computadora.
PORT=3000	El puerto en el que el servidor escuchará en producción (algunos servicios lo asignan automáticamente).
NODE_ENV=production	Le indica a Node.js y a muchas librerías que se está ejecutando en un entorno de producción, no de desarrollo.

J. ¿Qué debería ver en pantalla?

Al visitar la URL pública de tu aplicación desde cualquier dispositivo con internet (no solo tu computadora), EventHub carga y funciona igual que en localhost.

K. Error frecuente

⚠ Suponer que localhost y producción tienen la misma configuración, y descubrir hasta el final que faltan variables de entorno o que la base de datos de producción no está conectada.

L. Cómo encontrar el error

Revisa los logs de producción del servicio de hosting: la mayoría de errores de deployment (variables faltantes, dependencias no instaladas) aparecen ahí con detalle.

M. Ejercicio guiado

Publica una versión funcional de EventHub y escribe un README que explique cómo ejecutarlo y cómo desplegarlo.

N. Ejercicio independiente

Configura una variable de entorno distinta entre tu entorno local y el de producción (por ejemplo, un mensaje de bienvenida diferente), sin ver ningún ejemplo similar.

O. Preguntas de comprobación

[COMPRENSIÓN] ¿Qué diferencias aparecen entre un entorno local y uno de producción?

Respuesta de referencia: En producción aparecen usuarios reales, variables de entorno propias, seguridad, dominio, HTTPS, logs, disponibilidad y datos reales.

[COMPRENSIÓN] ¿Para qué sirve un archivo README en un proyecto publicado?

Respuesta de referencia: Explica a otras personas (y a ti mismo en el futuro) cómo instalar, ejecutar y desplegar el proyecto.

[APLICACIÓN] ¿Qué necesitarías configurar para que tu servidor de producción se conecte a la base de datos correcta?

Respuesta de referencia: Una variable de entorno DATABASE_URL apuntando a la base de datos de producción, no a la local.

[DIAGNÓSTICO] Tu app funciona en local pero en producción muestra un error 500 al cargar eventos. ¿Qué revisarías primero?

Respuesta de referencia: Los logs de producción, y si las variables de entorno (como DATABASE_URL) están correctamente configuradas en el servicio de hosting.

[TRANSFERENCIA] ¿Qué pasos adicionales necesitarías si tu aplicación además sirve imágenes subidas por los usuarios?

Respuesta de referencia: Investigar cómo el servicio de hosting maneja archivos persistentes, ya que muchos no guardan archivos subidos de forma permanente por defecto.

P. Mini reto

Escribe, sin ver ningún ejemplo, una checklist de 5 puntos que revisarías antes de anunciar que tu app "ya está en producción".

Q. Checkpoint

✓ **¿Puedes explicar de memoria qué cambia exactamente cuando pasas de localhost a producción?**

MÓDULO 20

Capstone EventHub Full-Stack

A. Lo que vas a aprender

Vas a integrar todo lo aprendido en una aplicación que puedas explicar y mostrar. Al terminar tendrás EventHub completo, defendible y publicado.

B. ¿Qué es?

El capstone es una prueba de transferencia: no consiste en copiar un tutorial entero. Debes tomar una especificación, diseñar la solución, construirla por etapas, probarla y defender las decisiones que tomaste.

C. ¿Por qué existe?

Saber seguir instrucciones paso a paso no es lo mismo que saber construir algo desde una especificación abierta. El capstone mide justamente eso: si puedes combinar todas las habilidades del curso frente a un problema que no es idéntico a ningún ejemplo anterior.

D. ¿Cómo funciona? (paso a paso)

- 1 Lees la especificación completa de EventHub antes de escribir una sola línea de código.
- 2 Diseñas la arquitectura (rutas, tablas, componentes) en papel primero.
- 3 Implementas por etapas, haciendo commits frecuentes.
- 4 Pruebas cada etapa antes de avanzar a la siguiente.
- 5 Revisas seguridad, despliegas y documentas en el README.

E. Diagrama visual

```
Requisitos -> Diseño -> Implementacion -> Pruebas -> Seguridad -> Deployment -> Demo -> README
```

F. Ejemplo cotidiano (analogía)

Los módulos anteriores son el entrenamiento. El capstone es el partido. El objetivo es demostrar que puedes combinar habilidades distintas para resolver un problema nuevo, no solo repetir ejercicios ya vistos.

G. Ejemplo dentro del proyecto EventHub

EventHub final: registro, login, roles, CRUD de eventos, inscripciones, validación, pruebas, README y una URL pública donde cualquiera pueda usarlo.

H. Código mínimo

TEXT

Requisitos -> Diseño -> Implementacion -> Pruebas
-> Seguridad -> Deployment -> Demo -> README

I. Explicación línea por línea

Requisitos	Lee y anota exactamente qué debe hacer la aplicación, sin asumir nada extra ni faltante.
Diseño	Dibuja la arquitectura: tablas, endpoints y componentes principales, antes de programar.
Implementación	Construye por etapas pequeñas, con commits frecuentes y descriptivos.
Pruebas	Verifica cada funcionalidad, incluyendo casos límite y de error.
Seguridad	Revisa validaciones, autorización y manejo de secretos.
Deployment	Publica la aplicación con una URL accesible desde internet.
Demo / README	Documenta cómo usar y desplegar el proyecto, listo para mostrarlo a otras personas.

J. ¿Qué debería ver en pantalla?

Una URL pública donde cualquier persona pueda registrarse, iniciar sesión, ver eventos, inscribirse, y donde un organizador pueda crear, editar y eliminar eventos.

K. Error frecuente

△ Copiar un tutorial completo (incluyendo el de este curso) y presentarlo como proyecto propio, sin poder explicar por qué se tomó cada decisión.

L. Cómo encontrar el error

Pídetle a ti mismo explicar en voz alta, sin ver el código, por qué elegiste cada tabla, cada endpoint y cada componente; si no puedes explicarlo, revísalo de nuevo.

M. Ejercicio guiado

Entrega EventHub con registro, login, roles, CRUD de eventos, inscripciones, validación, pruebas, README y URL pública.

N. Ejercicio independiente

Agrega una funcionalidad que no estaba en la especificación original (por ejemplo, favoritos o comentarios), diseñándola tú mismo de principio a fin.

O. Preguntas de comprobación

[COMPRENSIÓN] Explica de memoria el recorrido completo de una solicitud para crear un evento.

Respuesta de referencia: Una solicitud POST /eventos llega al backend, se valida, se procesa, se guarda en PostgreSQL y la respuesta vuelve al navegador.

[APLICACIÓN] ¿Qué decidirías primero al empezar el capstone: la base de datos o la interfaz visual?

Respuesta de referencia: Conviene empezar por diseñar la arquitectura general (datos y endpoints), ya que la interfaz depende de qué datos existen y cómo se piden.

[DIAGNÓSTICO] Tu capstone funciona en local, pero al mostrarlo a otra persona en su computadora falla. ¿Qué revisarías?

Respuesta de referencia: Si el proyecto depende de configuración local específica (variables de entorno, datos de prueba) que no está documentada en el README.

[TRANSFERENCIA] Si tuvieras que agregar pagos reales a EventHub, ¿qué módulos del curso reutilizarías como base?

Respuesta de referencia: Autenticación y autorización (para saber quién paga), persistencia (para guardar el estado del pago) y seguridad (para proteger datos sensibles).

[TRANSFERENCIA] ¿Qué parte de EventHub defenderías con más confianza en una entrevista de trabajo, y por qué?

Respuesta de referencia: Respuesta abierta: se espera que el estudiante identifique una decisión concreta que pueda explicar y justificar con detalle.

P. Mini reto

Antes de mostrar tu capstone a alguien más, escribe de memoria (sin ver el código) la lista completa de tablas, endpoints y componentes que construiste.

Q. Checkpoint

✓ **¿Puedes presentar EventHub en menos de 3 minutos, explicando qué hace, cómo está construido, y defender al menos dos decisiones técnicas?**

Qué hacer cuando algo no funciona

Guía general de debugging que aplica a cualquier módulo del curso.

1. Reproduce: Haz que el error vuelva a ocurrir de forma consistente.
2. Observa: Mira consola, Network, status code y datos reales.
3. Aísla: Reduce el problema a la pieza más pequeña posible.
4. Formula una hipótesis: Escribe qué crees que está pasando y por qué.
5. Prueba: Cambia una sola cosa y observa el efecto.
6. Corrige: Haz el cambio mínimo que resuelve la causa.
7. Verifica: Vuelve a ejecutar el caso original y una variante.
8. Documenta: Registra la causa y la solución si fue un error importante.

Frase clave: no adivines. Observa evidencia.

Evaluación del aprendizaje

La evaluación combina memoria, ejecución y transferencia. Una calificación alta exige demostrar que puedes hacer algo nuevo con las mismas ideas, no solo repetir lo visto.

Nivel	Evidencia	Aprobación sugerida
1. Comprender	Explica conceptos con tus propias palabras.	80% del quiz
2. Ejecutar	Completa ejercicios guiados sin errores fundamentales.	100% de checkpoints críticos
3. Resolver	Construye una solución sin copiar la implementación.	Funciona + explica decisiones
4. Transferir	Resuelve una variante nueva.	Al menos 70% sin ayuda
5. Integrar	Entrega un proyecto completo y defendible.	Todos los requisitos críticos

Checklist de graduación

- ☐ Sé explicar la diferencia entre frontend, backend, API y base de datos.
- ☐ Puedo crear una interfaz responsive con HTML semántico y CSS.
- ☐ Puedo programar funciones, condiciones, ciclos, arrays y objetos en JavaScript.
- ☐ Puedo manipular el DOM y manejar formularios.
- ☐ Puedo usar Git para guardar y recuperar cambios.
- ☐ Puedo consumir y construir una API REST.
- ☐ Puedo crear consultas SQL y relacionar tablas.
- ☐ Puedo conectar Express con PostgreSQL.
- ☐ Puedo explicar autenticación y autorización.
- ☐ Puedo identificar riesgos de seguridad básicos.
- ☐ Puedo depurar un error siguiendo evidencia.
- ☐ Puedo construir una interfaz con React.
- ☐ Puedo desplegar una aplicación y explicar su configuración.
- ☐ Puedo presentar EventHub y defender mis decisiones técnicas.

Glosario para principiantes

API

Interfaz con reglas que permite que un programa solicite o envíe datos a otro programa.

Backend

Código que corre en el servidor y procesa peticiones, reglas y datos.

Base de datos

Sistema que almacena información de forma estructurada y permite consultarla.

Branch

Línea de trabajo independiente en Git para desarrollar cambios sin afectar la rama principal.

Commit

Registro de un conjunto de cambios en Git.

DOM

Representación de la página que el navegador mantiene en memoria para poder manipularla con JavaScript.

Endpoint

Ruta específica de una API que recibe solicitudes y produce respuestas.

Frontend

Parte de la aplicación que el usuario ve y con la que interactúa.

HTTP

Protocolo que define cómo clientes y servidores intercambian solicitudes y respuestas en la web.

JSON

Formato de texto común para representar objetos y listas de datos.

Middleware

Función que procesa una solicitud antes de que llegue a la lógica final de una ruta.

Node.js

Entorno que permite ejecutar JavaScript fuera del navegador.

Repositorio

Carpeta gestionada por Git que contiene archivos y el historial de cambios.

Responsive

Diseño que se adapta a distintos tamaños y formas de pantalla.

REST

Estilo de diseño de APIs basado en recursos y operaciones HTTP coherentes.

Runtime

Entorno que proporciona lo necesario para ejecutar un programa.

Servidor

Sistema que recibe solicitudes y devuelve respuestas o servicios a clientes.

SQL

Lenguaje utilizado para consultar y modificar bases de datos relacionales.

UI

Interfaz de usuario: lo que una persona ve y utiliza.

UX

Experiencia de usuario: facilidad, claridad y calidad con que una persona logra un objetivo.

Capstone: EventHub

Especificación mínima: registro, login, roles, listado de eventos, detalle, crear/editar/eliminar, inscripción a eventos, validación, errores, pruebas, README y despliegue. Debes ser capaz de reconstruir la arquitectura en una hoja antes de comenzar.

Examen final de transferencia

Responde primero sin mirar apuntes.

- 1 Explica el recorrido de una petición POST /eventos, de principio a fin.
- 2 ¿Por qué el navegador no debería conectarse directamente a PostgreSQL?
- 3 ¿Qué diferencia hay entre autenticación y autorización?
- 4 ¿Cómo investigarías un error 500 en producción?
- 5 ¿Qué cambiarías si la lista de eventos funciona en local pero falla en producción?
- 6 ¿Qué pruebas harías antes de publicar una nueva funcionalidad?